4 — Linear models

Machine Learning (MPA-MLR)

Tomáš Vičar
Department of Biomedical Engineering · FEEC, Brno University of Technology

2026

1 · One equation, two tasks

> One dot product of weights and features predicts a number or a class — everything else is the choice of loss.

Linear models: regression vs. classification

One output of last lecture's $\mathbf{z} = W^\top \mathbf{x}$ — the **score**:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$$

Bias trick: an **affine** $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ becomes purely linear — prepend $x_0 = 1$ to $\mathbf{x}$, put $w_0 = b$ into $\mathbf{w}$:

$$\tilde{\mathbf{x}} = \begin{bmatrix} 1 \\ \mathbf{x} \end{bmatrix}, \quad \tilde{\mathbf{w}} = \begin{bmatrix} b \\ \mathbf{w} \end{bmatrix} \;\Rightarrow\; f(\mathbf{x}) = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$$

We write it **with or without** b, whichever reads better.

Linear regression — keep the score

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} \in \mathbb{R}$$

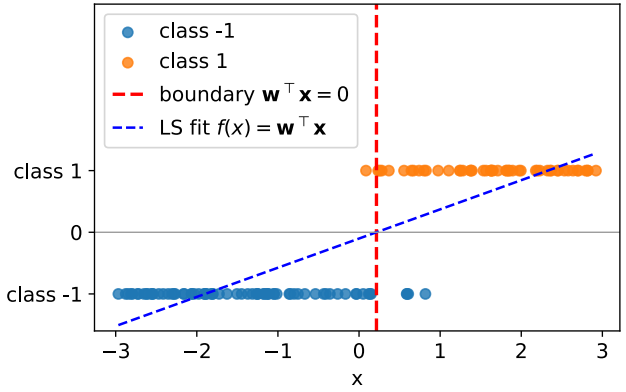
Linear classification (2 classes) — take its sign

$$\hat{y} = \operatorname{sign}(\mathbf{w}^\top \mathbf{x} + b) = \operatorname{sign}(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}) \in \{-1, +1\}$$

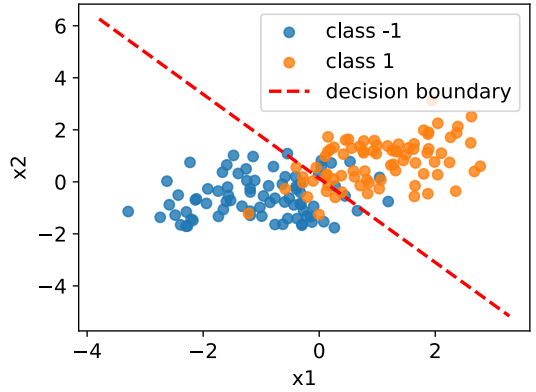
Decision boundary: $\mathbf{w}^\top \mathbf{x} + b = 0$

Linear classification: visualizations

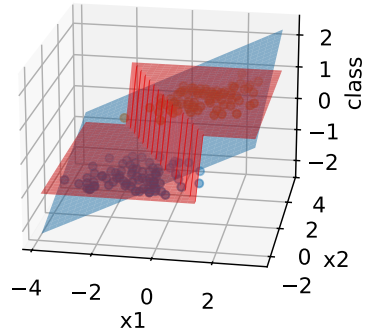
1D classification



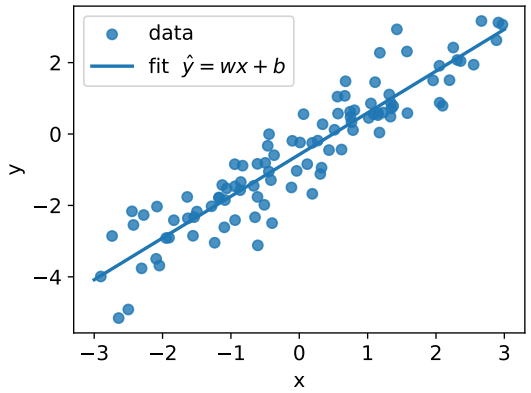
2D classification



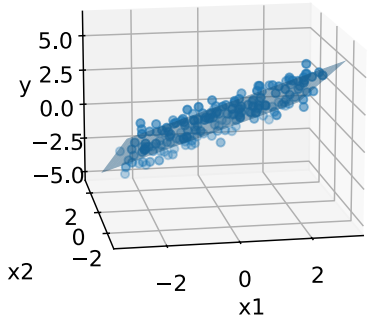
2D classification



1D regression



2D regression



The same equation everywhere: a **line** in 1D, a **plane** in 2D, a **hyperplane** in d dimensions. Regression uses the score, classification its **sign**.

Fit a linear model

Data: $\{(\mathbf{x}_i, y_i)\}_{i=1}^n,\ \mathbf{x}_i \in \mathbb{R}^d,\ y_i \in \mathbb{R}$ (regression) or $y_i \in \{-1, +1\}$ (classification)

Prediction: $\hat{y} = \mathbf{w}^\top \mathbf{x}$ or $\hat{y} = \operatorname{sign}(\mathbf{w}^\top \mathbf{x})$ — the only thing to learn is $\mathbf{w}$.

Training: minimize the average loss + regularization

$$\min_{\mathbf{w}} \underbrace{\frac{1}{n}\sum_{i=1}^{n} \mathcal{L}\big(y_i,\ \mathbf{w}^\top \mathbf{x}_i\big)}_{\text{fit the data}} + \underbrace{\lambda\, \Omega(\mathbf{w})}_{\text{keep } \mathbf{w} \text{ small}}$$

Loss $\mathcal{L}$

regression: MSE, MAE

classification: SVM, perceptron, LDA, logistic (log-loss)

Regularizer Ω

$\|\mathbf{w}\|_2^2$ — Ridge (L_2)

$\|\mathbf{w}\|_1$ — LASSO (L_1)

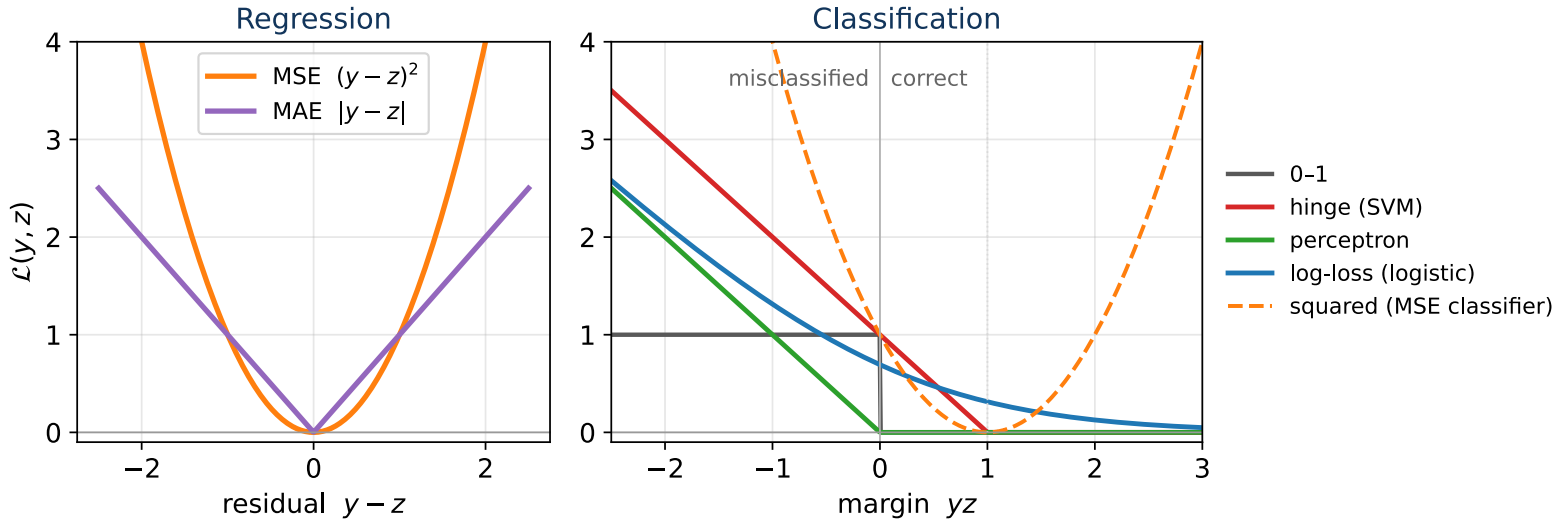
Weight λ

fit ↔ simplicity

(a hyperparameter)

Loss choices for linear models

Score $z = \mathbf{w}^\top \mathbf{x}$ · regression losses depend on the **residual** $y - z$, classification losses on the **margin** yz ($yz > 0 \Leftrightarrow$ correct).



- **MSE** $(y-z)^2$ — smooth, closed form, sensitive to outliers
- **MAE** $|y-z|$ — robust to outliers; no closed form (LP/IRLS)

- **SVM** — max-margin, *hinge* $\max(0,\ 1-yz)$
- **Perceptron** — $\max(0,\ -yz)$
- **Logistic** — log-loss $\log\left(1+e^{-yz}\right)$
- **LDA** — Fisher criterion (no per-sample loss)

2 · Linear regression

> From a line through two points to ordinary least squares, robust fits and regularization.

Linear regression with two points

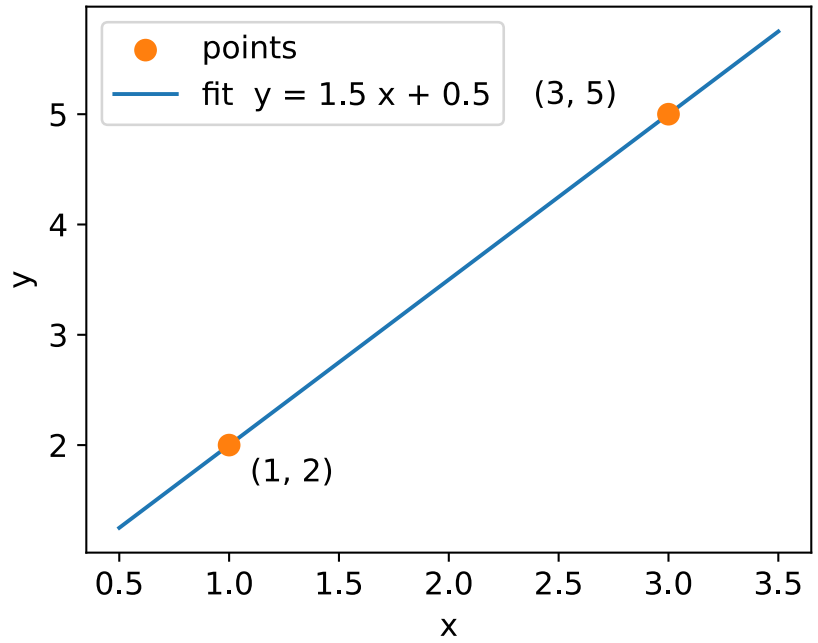
Model: $y = w\,x + b$ · **two equations, two unknowns:**

$$\begin{aligned} y_1 &= w\,x_{11} + b \\ y_2 &= w\,x_{21} + b \end{aligned} \quad \Longleftrightarrow \quad \begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & x_{11} \\ 1 & x_{21} \end{bmatrix}}_{X} \underbrace{\begin{bmatrix} b \\ w \end{bmatrix}}_{\tilde{\mathbf{w}}}, \qquad \mathbf{y} = X\,\tilde{\mathbf{w}}$$

Solution: $\boxed{\tilde{\mathbf{w}} = X^{-1}\mathbf{y}}$

Prediction for a new input x_{new}, or for many new samples in the matrix X_{new}:

$$\hat{y}_{\text{new}} = w\,x_{\text{new}} + b, \qquad \hat{\mathbf{y}}_{\text{new}} = X_{\text{new}}\,\tilde{\mathbf{w}}$$



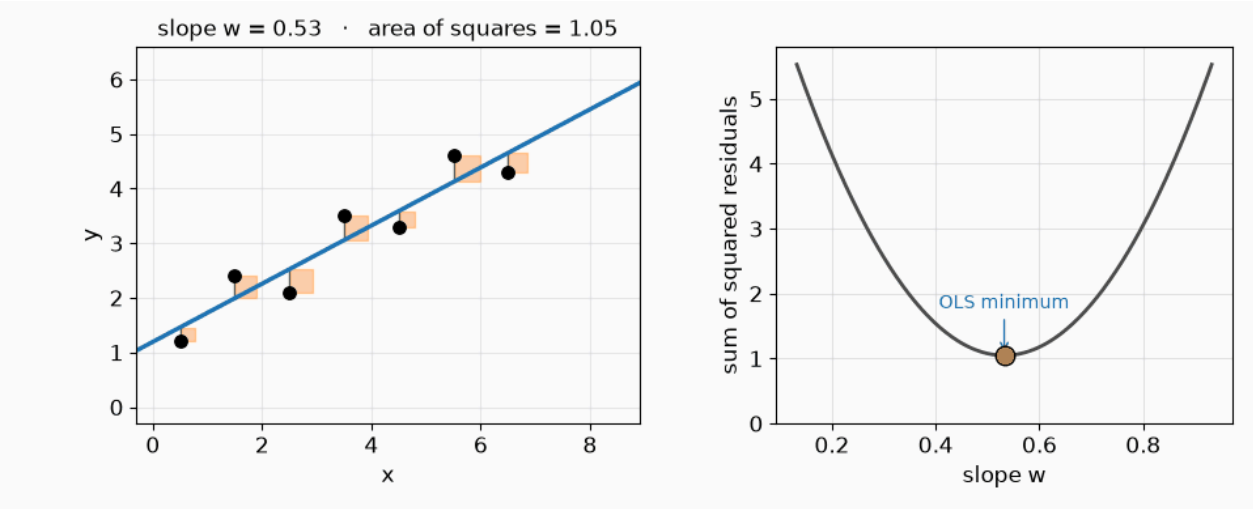
By hand for $(1,2),(3,5)$: $X = \begin{bmatrix} 1 & 1 \\ 1 & 3 \end{bmatrix}, \det X = 2, X^{-1} = \frac{1}{2}\begin{bmatrix} 3 & -1 \\ -1 & 1 \end{bmatrix}, \tilde{\mathbf{w}} = \frac{1}{2}\begin{bmatrix} 3\cdot 2 - 5 \\ -2 + 5 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 1.5 \end{bmatrix}$

Two points → a line through both. **Three or more** → no such line: make $X\tilde{\mathbf{w}}$ **as close as possible** to $\mathbf{y}$.

MSE linear regression (OLS)

Ordinary least squares: predictions $X\mathbf{w}$ as close as possible to $\mathbf{y}$ in the MSE (L_2) sense — the **total area of the squares**:

$$\underset{\mathbf{w}}{\text{minimize}} \;\; g(\mathbf{w}) = \sum_{i=1}^{n} (\mathbf{x}_i^\top \mathbf{w} - y_i)^2 = \|X\mathbf{w} - \mathbf{y}\|_2^2$$



Convex, unconstrained → one minimum, where the gradient is zero (gradient descent would work too):
$\nabla g(\mathbf{w}) = 2X^\top(X\mathbf{w} - \mathbf{y}) = \mathbf{0}$

Closed form: $\boxed{\mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y} = X^\dagger \mathbf{y}}$ · $X^\dagger$ — the Moore–Penrose **pseudoinverse** · derivation in the appendix

OLS by hand: three points

Data: $(0,1), (1,2), (2,2)$ — no line passes through all three.

1 · Build X (column of ones = bias trick) and $\mathbf{y}$:

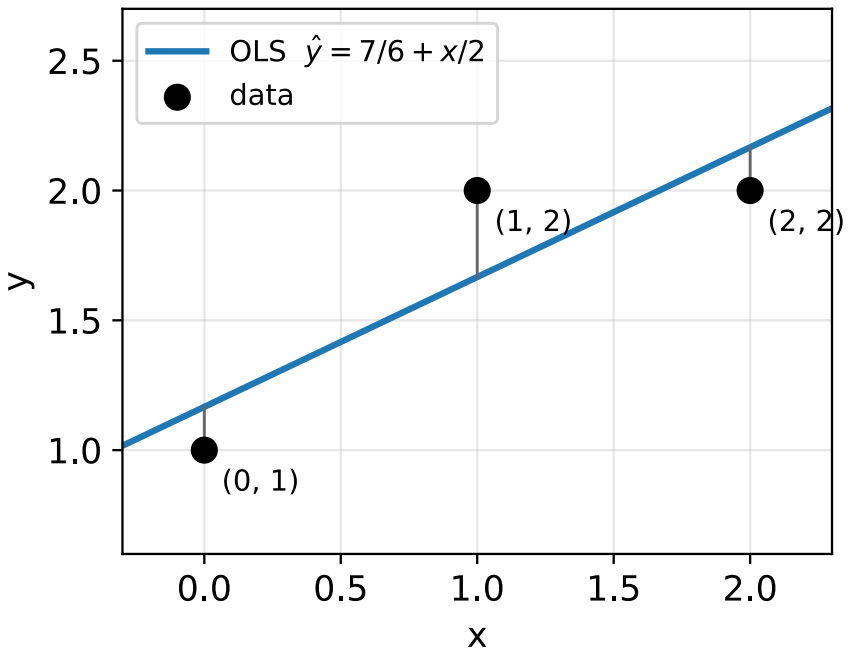
$$X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}$$

2 · Two small products:

$$X^\top X = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}, \quad X^\top \mathbf{y} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

3 · Solve the 2 × 2 system:

$$\tilde{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y} = \tfrac{1}{6} \begin{bmatrix} 5 & -3 \\ -3 & 3 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 7/6 \\ 1/2 \end{bmatrix} \Rightarrow \hat{y} = \tfrac{7}{6} + \tfrac{1}{2} x$$



Residuals $\left(-\frac{1}{6}, \frac{1}{3}, -\frac{1}{6}\right)$, sum of squares $\frac{1}{6}$ — the smallest any line can achieve.

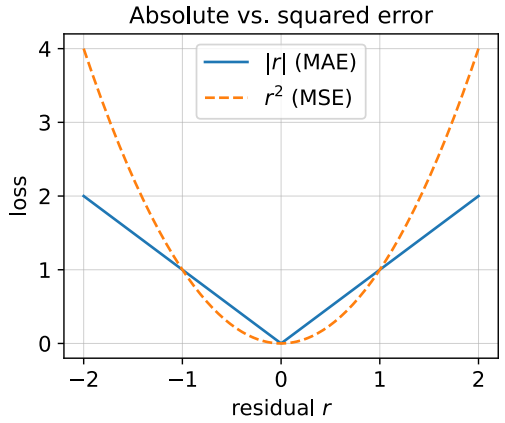
The same three points with ridge ($\lambda = 1$) by hand — in the appendix

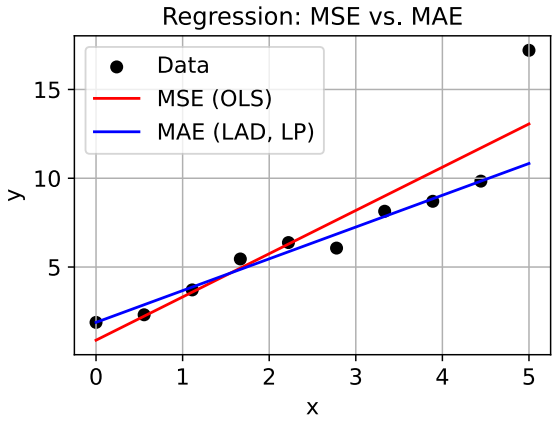
Linear regression with MAE (LAD)

Least absolute deviations — the same model, the absolute instead of the squared error:

$$\underset{\mathbf{w}}{\text{minimize}} \quad \sum_{i=1}^{n} |\mathbf{x}_i^\top \mathbf{w} - y_i| = \|X\mathbf{w} - \mathbf{y}\|_1$$

No closed form — solved by optimization, e.g. as a linear program (LP formulation in the appendix).

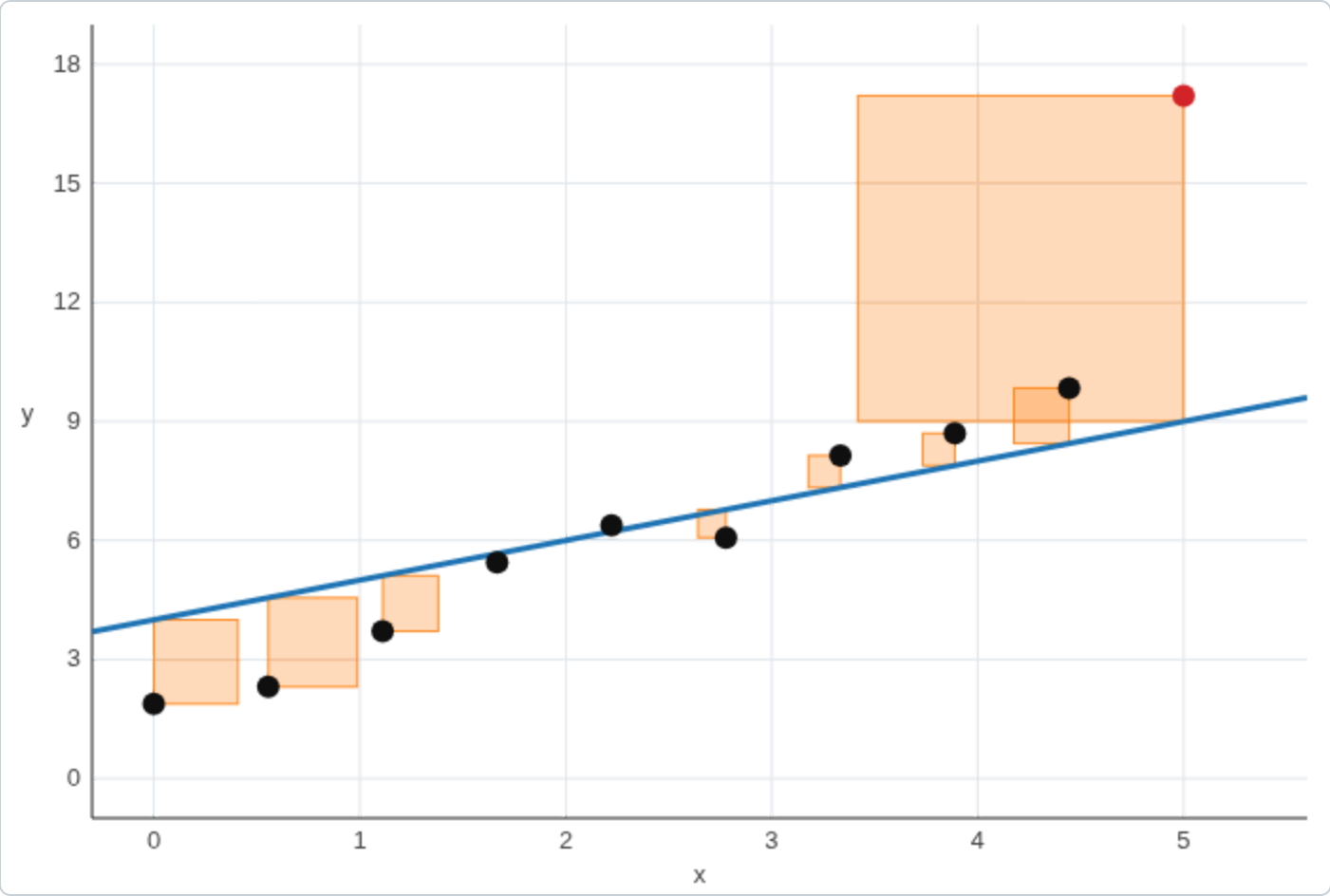




- **Large** errors: $L_1 \ll L_2$ → an outlier cannot dominate → **robust**
- **Small** errors are still penalized: $0.01^2 = 0.0001$ is practically ignored by L_2, not by L_1

$$\|\mathbf{x}\|_2 = \|\mathbf{x}\| = \sqrt{\textstyle\sum_i x_i^2} \cdot \|\mathbf{x}\|_2^2 = \mathbf{x}^\top\mathbf{x} = \textstyle\sum_i x_i^2 \cdot$$
$$\|\mathbf{x}\|_1 = \textstyle\sum_i |x_i|$$

Playground: fit a line by hand



Move the line. Orange **squares** = squared errors (MSE), purple **bars** = absolute errors (MAE).

slope **w** = 1.00

intercept **b** = 4.00

☑ squares ☐ bars

☑ outlier (last point +6)

best MSE (OLS) | best MAE (LAD)

your line: MSE **8.27**, MAE **1.81**
best MSE (OLS): 2.80 at w = 2.44, b = 0.88
best MAE (LAD): 0.94 at w = 1.79, b = 1.88

Ridge vs. LASSO regularization for linear regression

Ridge regression (L_2)

$$\underset{\mathbf{w}}{\text{minimize}} \; \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

- **Shrinks** all coefficients smoothly
- Keeps all features (**no exact zeros**)
- **Closed form:** $\mathbf{w} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$

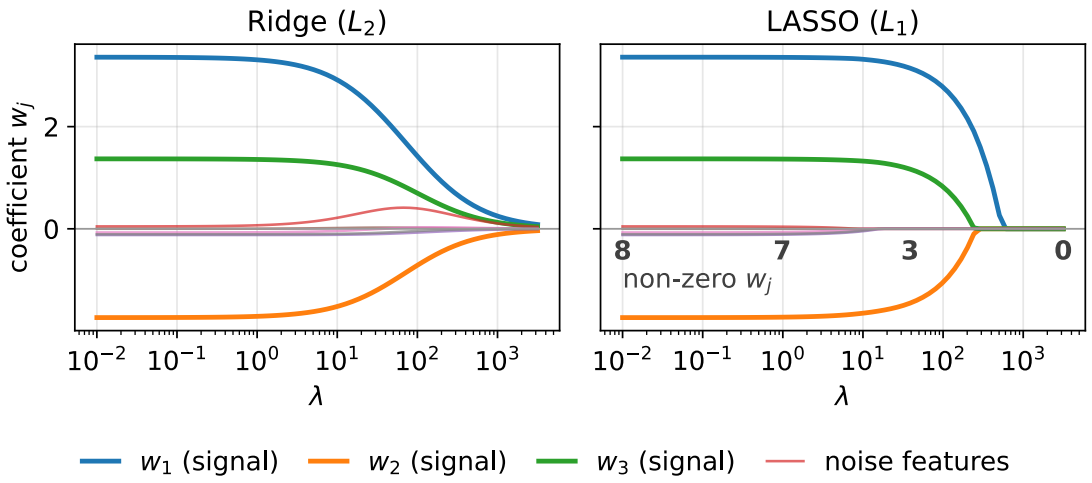
- λ — a **hyperparameter**: fit $\leftrightarrow$ regularization
- LASSO: $\uparrow \lambda \rightarrow$ **fewer features** selected
- The same penalties work for other methods too (LAD, linear classifiers)
- Penalties depend on the feature **scale** → **standardize** first (second lecture)

Appendix: why the L_1 corner gives zeros · ridge on three points by hand

LASSO regression (L_1)

$$\underset{\mathbf{w}}{\text{minimize}} \; \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1$$

- **Feature selection:** some $w_j = 0$ exactly
- Robust to irrelevant features
- **No closed form** — convex optimization



3 · Nonlinear features and kernels

> The model stays linear in the parameters — we only change what we feed into it.

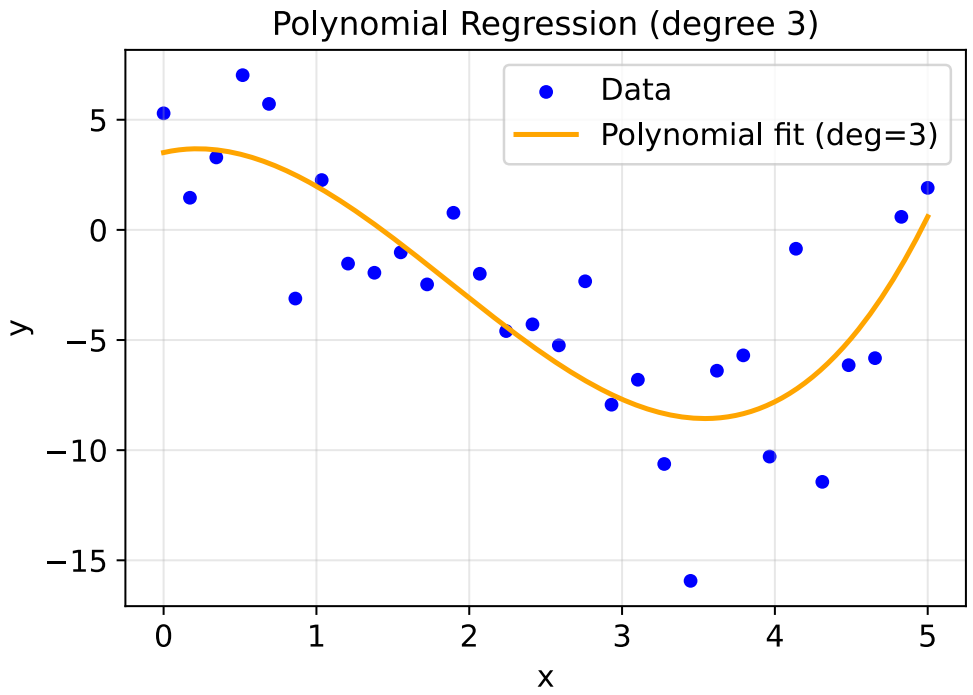
Polynomial regression

- **Idea:** feed powers of x into the same linear model

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_m x^m$$

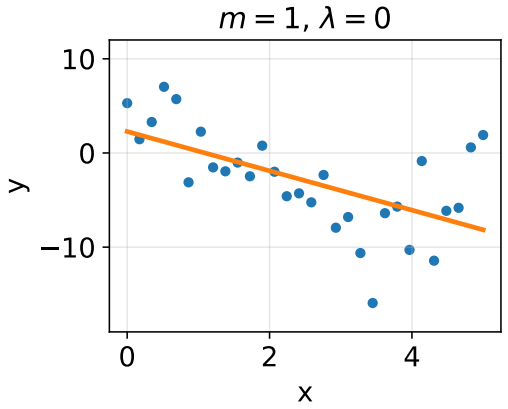
- **Nonlinear in x, linear in $\mathbf{w}$** → OLS and ridge work unchanged.
- Only the **matrix X changes** — its columns are powers: $\mathbf{y} \approx X\mathbf{w}$,

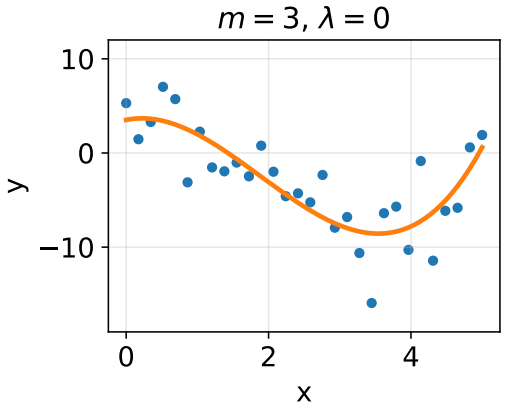
$$X = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^m \\ 1 & x_2 & x_2^2 & \cdots & x_2^m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^m \end{bmatrix}$$

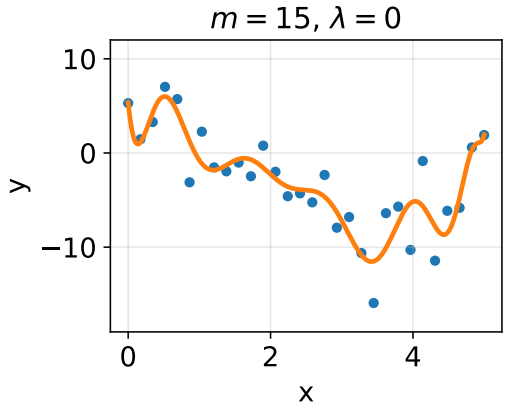


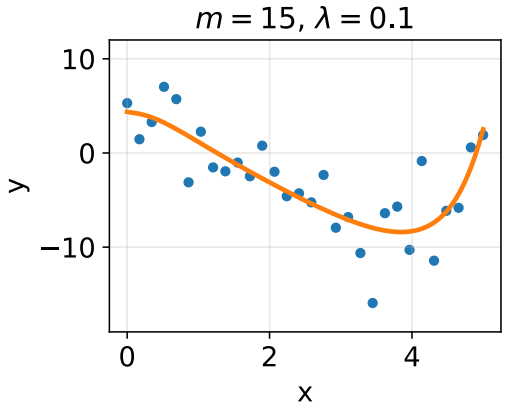
Code example · **Explicit** expansion computes the new columns; kernels do it **implicitly** via the dual form (as kernel PCA in the third lecture).

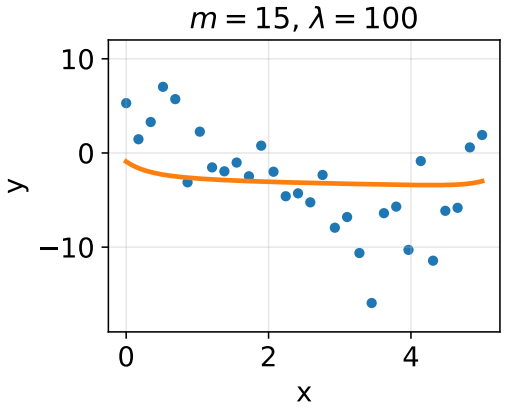
Effect of polynomial degree and regularization











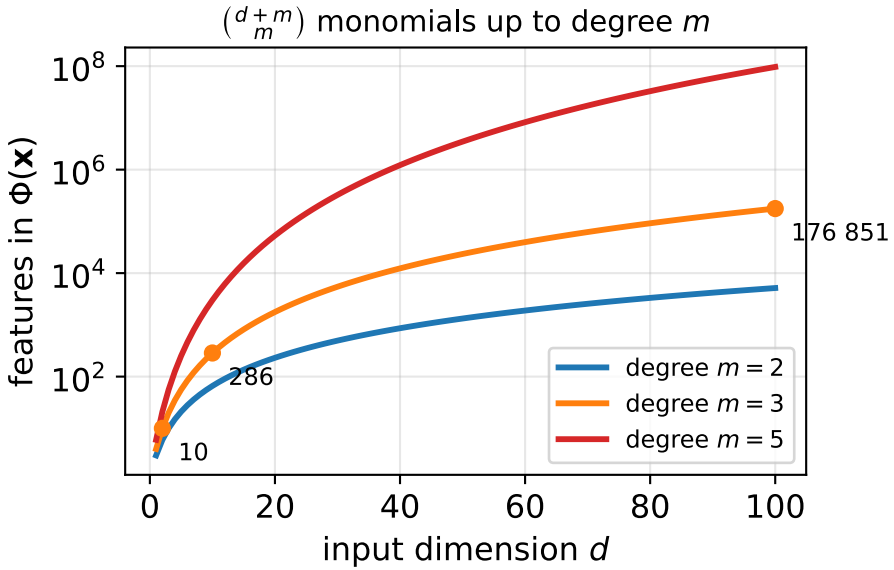
Two hyperparameters: the **degree** m (top row, $\lambda = 0$) and the **regularization** λ (bottom row, $m = 15$; its $\lambda = 0$ is the top-right panel).

Which one is an example of over-fitting and which one of under-fitting?

Explicit expansion explodes

Degree 3 in 2D already has 10 features:

$$\Phi^{(3)}(\mathbf{x}) = [\,1,\ x_1,\ x_2,\ x_1^2,\ x_1x_2,\ x_2^2,\ x_1^3,\ x_1^2x_2,\ x_1x_2^2,\ x_2^3\,]^\top$$



Monomials up to degree m in d variables: $\binom{d+m}{m}$

- $d = 2,\ m = 3$: $\binom{5}{3} = 10$
- $d = 10,\ m = 3$: 286
- $d = 100,\ m = 3$: $176\,851$
- **RBF kernel:** infinite-dimensional Φ

Problem: computing $\Phi(\mathbf{x})$ is **expensive or impossible**.

Primal vs. dual ridge regression

Primal — learn d weights $\mathbf{w}$

$$\min_{\mathbf{w}} \ \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda\|\mathbf{w}\|_2^2$$

$$\mathbf{w}^* = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$$

$$\hat{y}_{\text{new}} = \mathbf{x}_{\text{new}}^\top \mathbf{w}$$

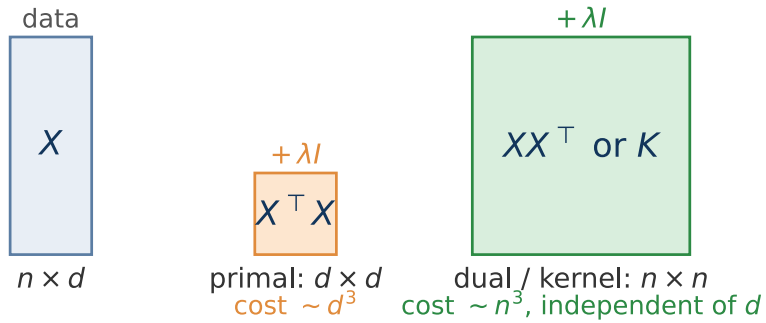
Dual — learn n coefficients $\boldsymbol{\alpha}$, one per sample

$$\min_{\boldsymbol{\alpha}} \ \tfrac{1}{2}\,\boldsymbol{\alpha}^\top (XX^\top + \lambda I)\boldsymbol{\alpha} - \mathbf{y}^\top \boldsymbol{\alpha}$$

$$\boldsymbol{\alpha}^* = (XX^\top + \lambda I)^{-1}\mathbf{y}, \qquad \mathbf{w} = X^\top \boldsymbol{\alpha}$$

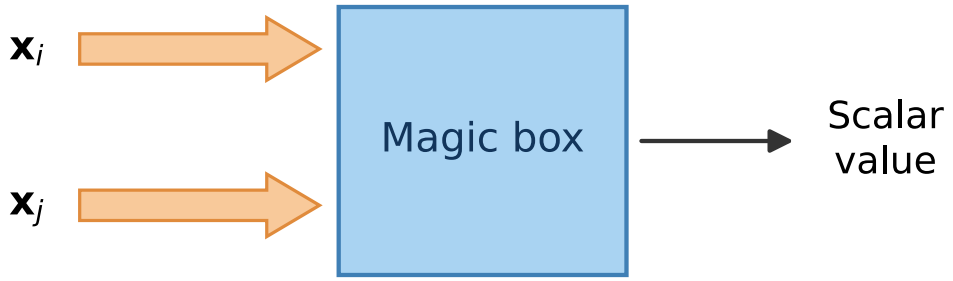
$$\hat{y}_{\text{new}} = \textstyle\sum_i \alpha_i \, \mathbf{x}_i^\top \mathbf{x}_{\text{new}}$$

In the dual the data appear **only as dot products** (red): $XX^\top$ holds $\mathbf{x}_i^\top \mathbf{x}_j$ for all pairs, the prediction needs $\mathbf{x}_i^\top \mathbf{x}_{\text{new}}$.



Appendix: derivation · all formulas. Same problem, same prediction. **Many samples, few features** → primal; **few samples, many features** → dual.

Kernel trick — recap from the third lecture



Kernel = dot product in the feature space, **without computing Φ**:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$$

Check by hand: $\Phi(\mathbf{x}) = (x_1^2,\ x_2^2,\ \sqrt{2}\, x_1 x_2)$, $\mathbf{x}_i = (1, 2), \mathbf{x}_j = (3, 1)$

- $\Phi(\mathbf{x}_i) = (1,\ 4,\ 2\sqrt{2})$, $\Phi(\mathbf{x}_j) = (9,\ 1,\ 3\sqrt{2})$
- **explicit:** $\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j) = 9 + 4 + 12 = 25$
- **kernel:** $(\mathbf{x}_i^\top \mathbf{x}_j)^2 = (3 + 2)^2 = 25$ ✓
- **degree 3:** $(\mathbf{x}_i^\top \mathbf{x}_j + 1)^3$ contains all 10 monomials of $\Phi^{(3)}$ (with weights)

Works whenever the algorithm uses the samples **only through dot products $\mathbf{x}_i^\top \mathbf{x}_j$** — its **dual form**.

Why it holds for every pair: the algebra in the appendix

Kernel function and kernel matrix

Common kernels:

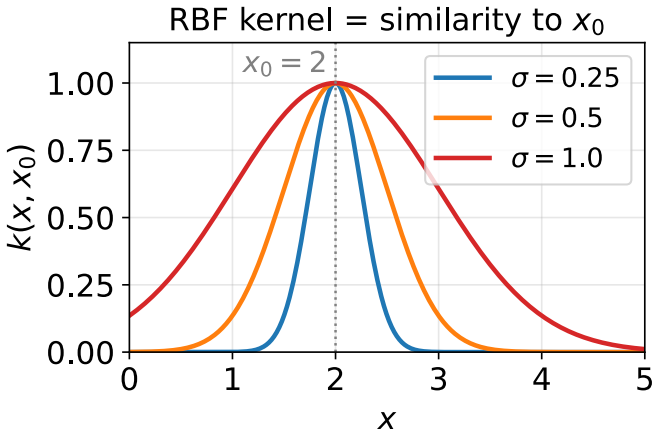
- **Linear:** $k(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$
- **Polynomial:** $k(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j + c)^p$
- **RBF (Gaussian):**

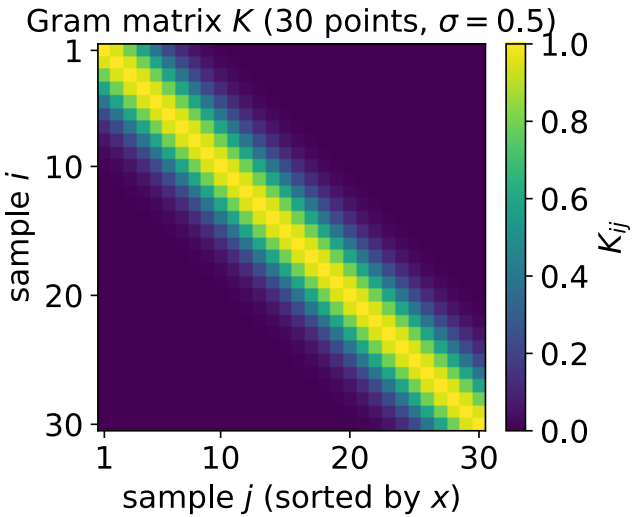
$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}\right)$$

Kernel (Gram) matrix for n samples:

$$K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j), \qquad K \in \mathbb{R}^{n \times n}$$

> $k(\mathbf{x}_i, \mathbf{x}_j)$ **replaces** $\mathbf{x}_i^\top \mathbf{x}_j$, and K **replaces** $XX^\top$.





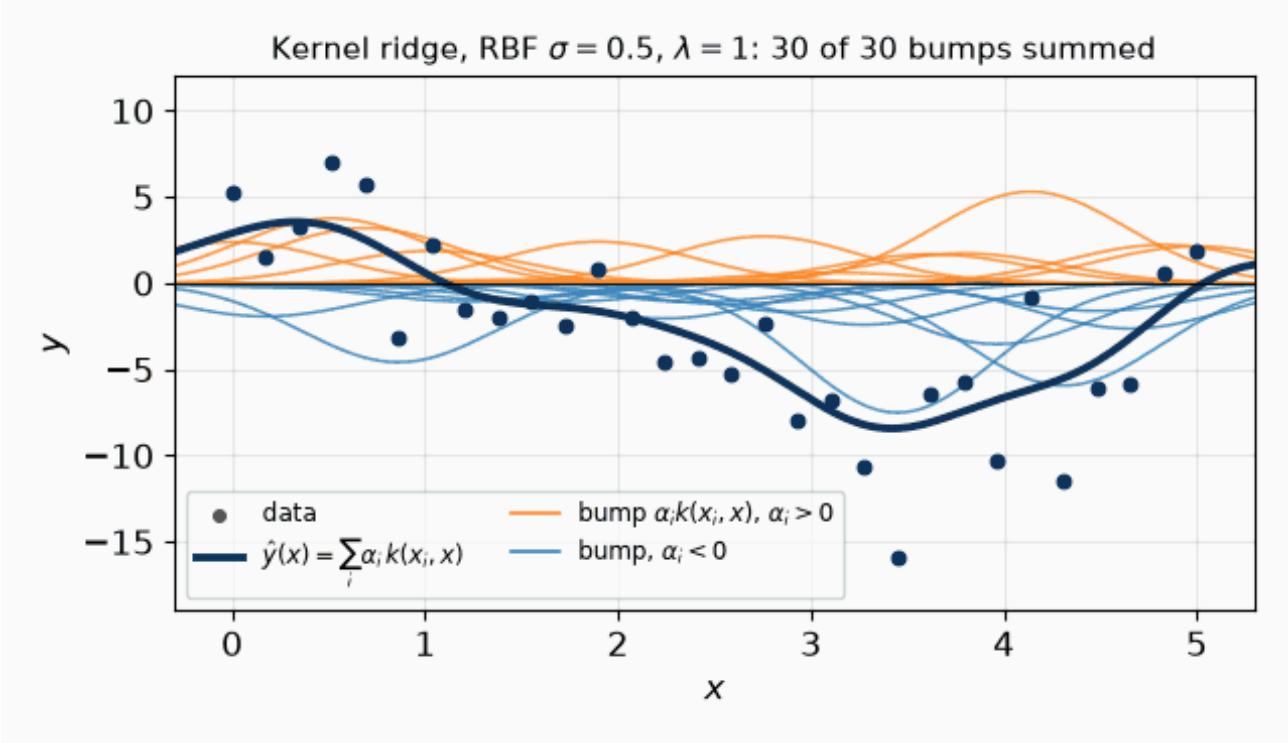
Kernel ridge regression (KRR)

Take the dual and **swap the dot products**:

- $XX^\top \;\to\; K$
- $\mathbf{x}_i^\top \mathbf{x}_{\text{new}} \;\to\; k(\mathbf{x}_i, \mathbf{x}_{\text{new}})$

$$\boldsymbol{\alpha}^* = (K + \lambda I)^{-1}\mathbf{y}$$

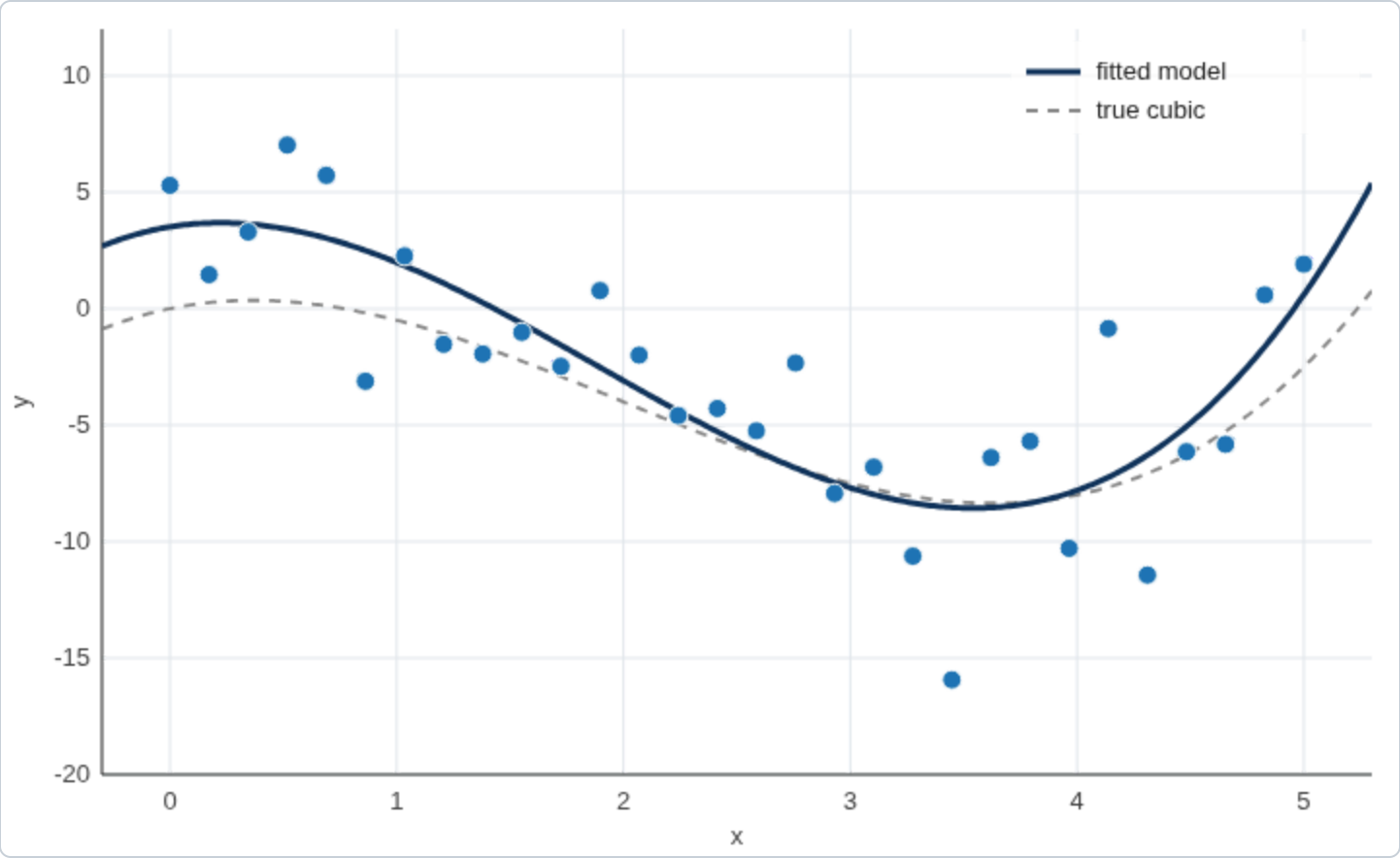
$$\hat{y}_{\text{new}} = \sum_{i=1}^{n} \alpha_i \, k(\mathbf{x}_i, \mathbf{x}_{\text{new}})$$



- **Same result** as the explicit Φ, **cost independent of its dimension**.
- **Price:** an $n \times n$ matrix.

code example

Playground: polynomial features vs. kernel ridge



Model

- (●) polynomial features (explicit)
- ( ) RBF kernel ridge (implicit)

degree m = 3

ridge λ = 0

☑ true curve ☑ bumps $\alpha_i\, k(x_i, x)$ reset

Parameters: 4 weights w_j, largest |coefficient| = 11.9

training MSE = 8.91

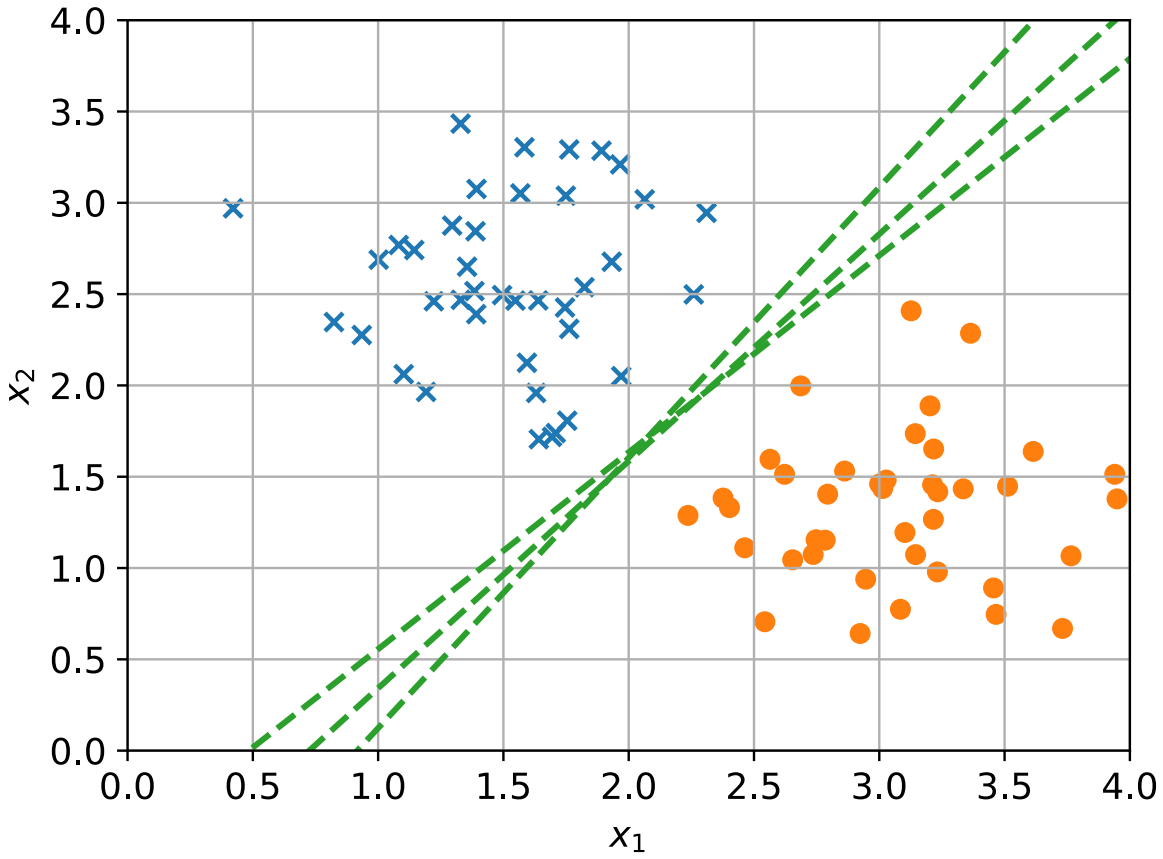
distance from the true curve (RMSE on [0, 5]) = 1.78

The 30 points of the polynomial slides (a cubic plus noise). Polynomial: ridge on the powers of x rescaled to [−1, 1], intercept not penalized. Kernel: $\alpha = (K + \lambda I)^{-1} y$.

4 · Linear classification

> Same line, different question: not how far, but on which side.

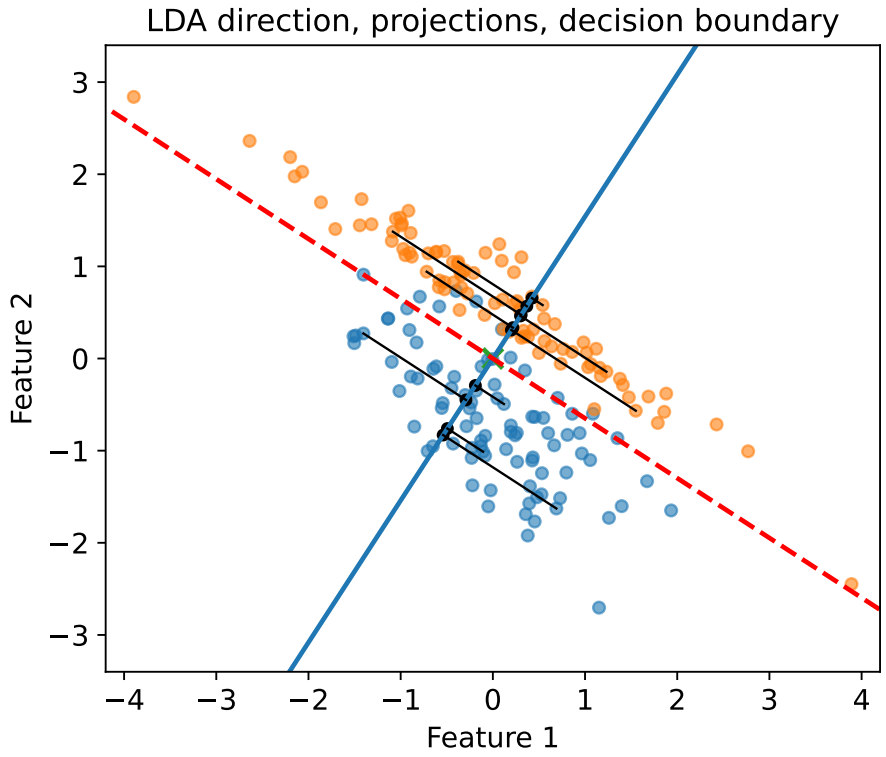
Linearly separable data

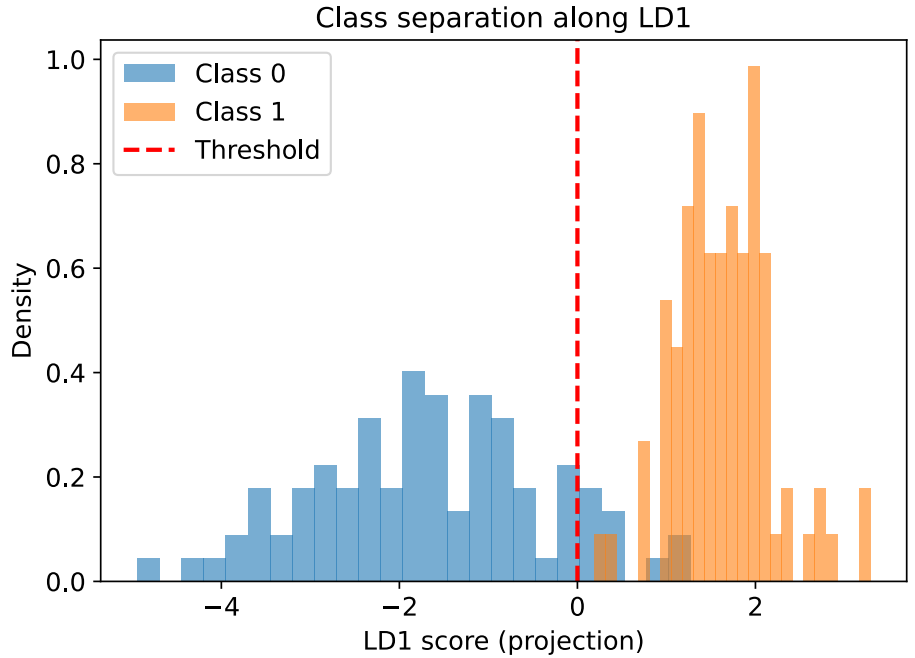


- **Separable:** some line (a hyperplane in d dimensions) puts *every* sample on its correct side.
- **Not unique:** all three lines make zero errors. Which one is best? → SVM.
- **Real data:** noise, overlap, wrong labels ⇒ almost never exactly separable.
- **Still needed** for perceptron convergence and for SVM theory.

LDA as a classifier — recap

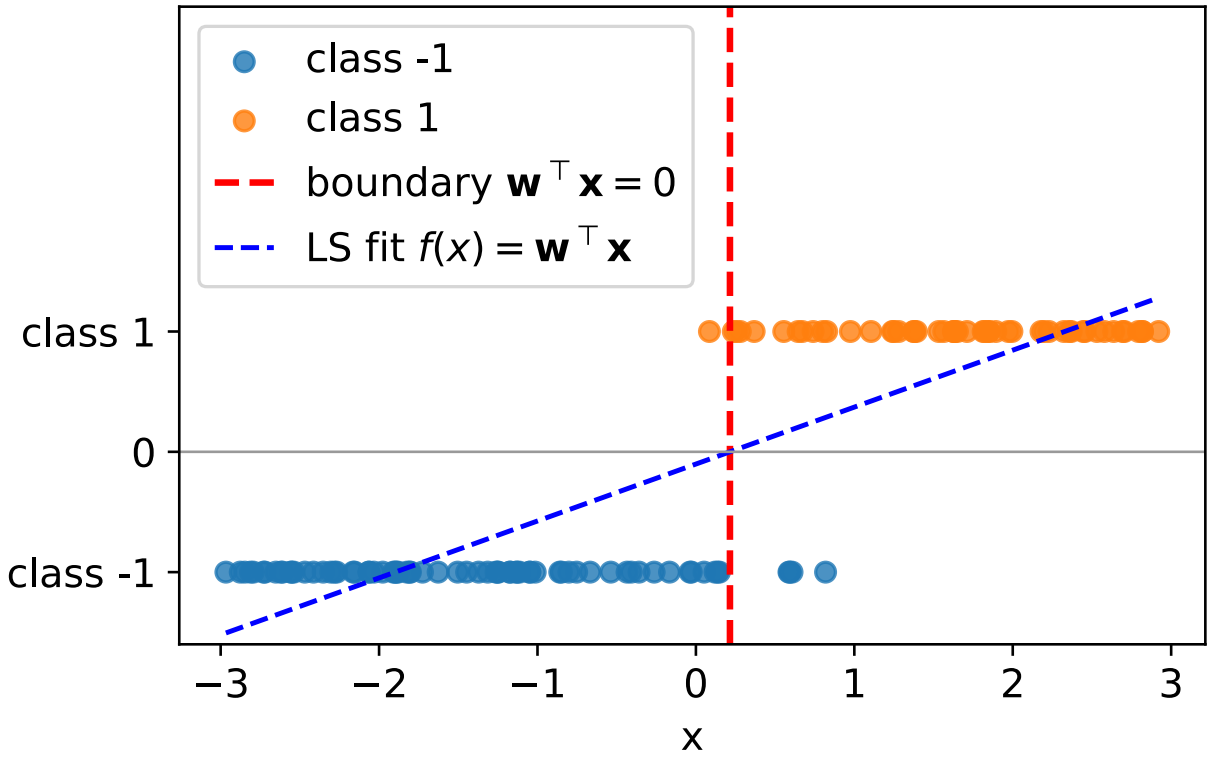
- **Direction:** $\mathbf{w} = S_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$ — class means far apart, within-class spread small (third lecture).
- **Decision:** compare the score $\mathbf{w}^\top \mathbf{x}$ with a threshold t^*; equal priors → t^* = midpoint of the projected class means.
- **No per-sample loss:** only class means and scatter enter — unlike the other classifiers today.





Fisher criterion and the threshold with unequal priors — appendix

MSE classifier



- **Train:** ordinary least squares on labels $y \in \{-1, +1\}$ — simple, closed form.
- **Predict:** $\hat{y} = \operatorname{sign}(\mathbf{w}^\top \mathbf{x} + b)$.
- **Problem:** correct points far from the boundary still pull the line — they are asked for 1, not 3.
- ⇒ **sensitive to outliers**, often **poor accuracy**.

Logistic regression

Idea: squeeze the score $z = \mathbf{w}^\top \mathbf{x} + b$ into $(0, 1)$ (or $(-1, 1)$) with a sigmoid.

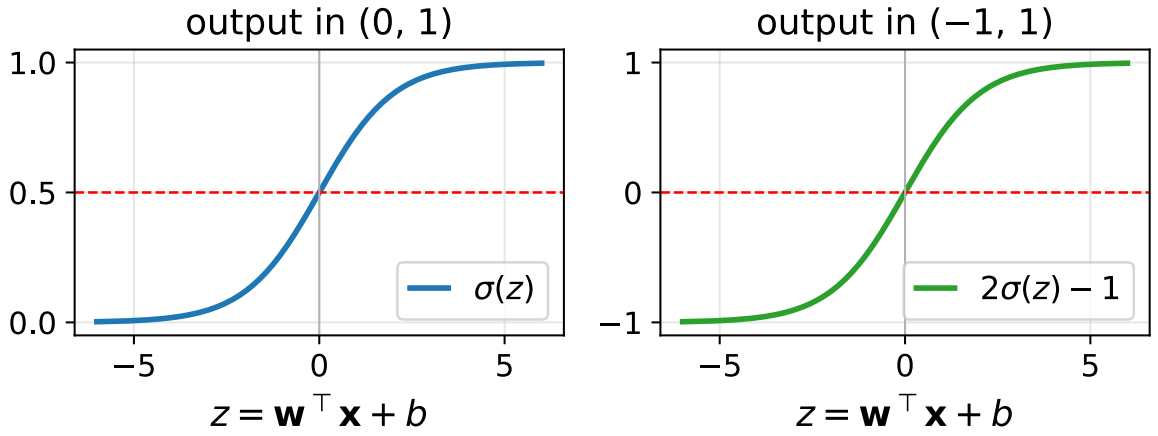
Model (probabilistic):

$$p(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Decision rule:

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \iff z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

equivalently $\hat{y} = \operatorname{sign}(z)$ for $y \in \{-1, +1\}$.



Loss (cross-entropy / logistic) — one function, two label codings:

$$\mathcal{L}_{\log}(y, z) = \begin{cases} -\,y \log \sigma(z) - (1 - y) \log(1 - \sigma(z)) & \text{for } y \in \{0, 1\}, \\ \log\left(1 + e^{-yz}\right) & \text{for } y \in \{-1, +1\}. \end{cases}$$

Third lecture: logistic regression on the rings — 0.55 in 2D, 1.00 with $x_1^2 + x_2^2$ added. · Appendix: the two forms agree; a gradient step by hand

Logistic regression: a probabilistic model

Labels $y \in \{0, 1\}$ (the first loss form of the previous slide); $z_i = \mathbf{w}^\top \mathbf{x}_i + b, p_i = \sigma(z_i)$.

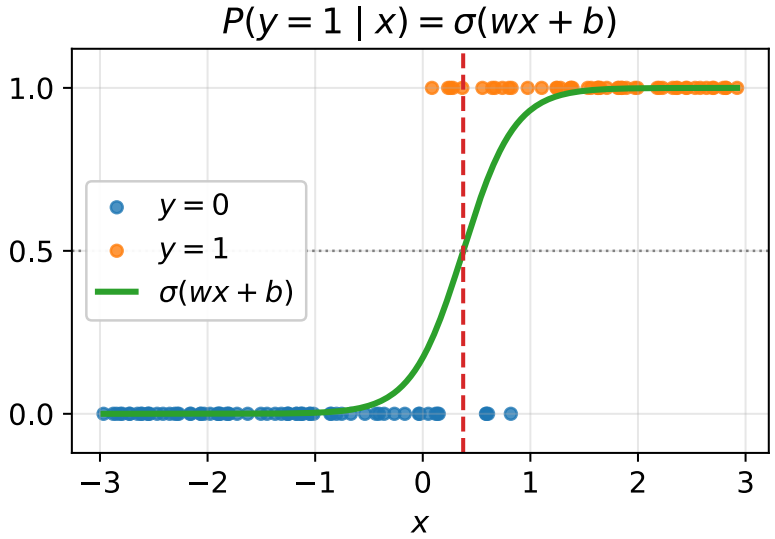
- **Model:** $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$
- **Log-odds are linear** in $\mathbf{x}$:

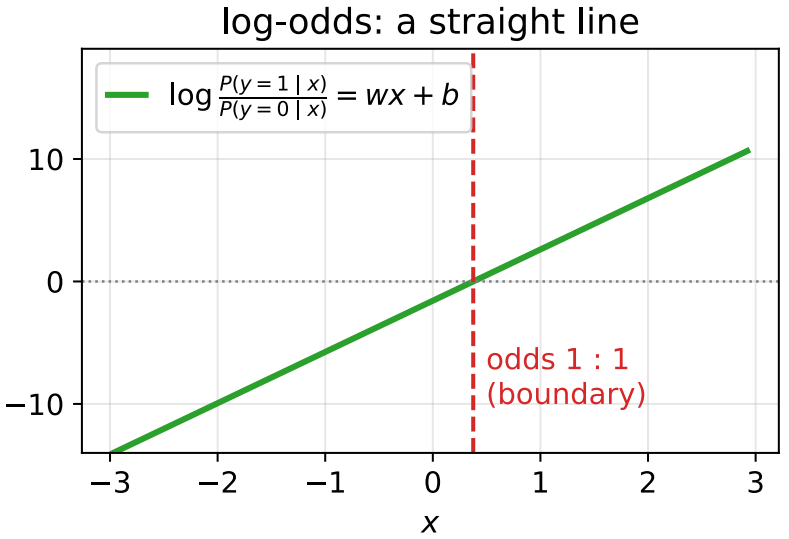
$$\log \frac{P(y = 1 \mid \mathbf{x})}{P(y = 0 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x} + b$$

- **Likelihood** of the training labels:

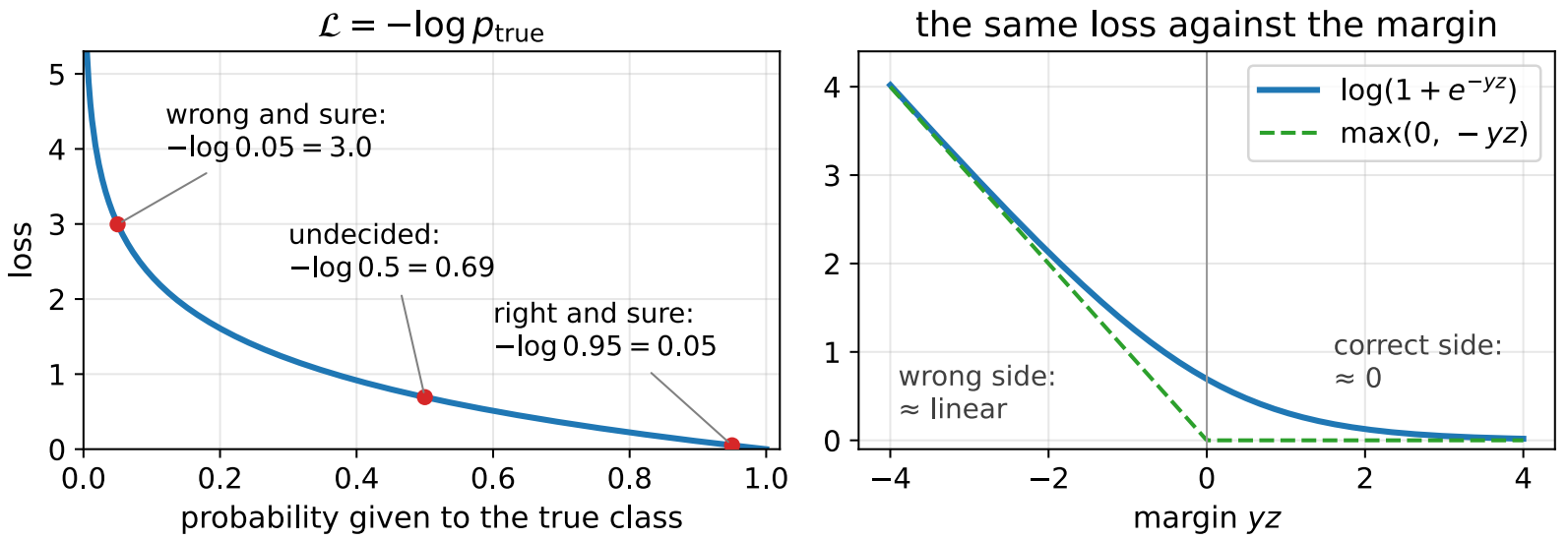
$$L(\mathbf{w}, b) = \prod_{i=1}^{n} p_i^{y_i} \, (1 - p_i)^{1 - y_i}$$

- **Maximum likelihood** $\Leftrightarrow$ minimize $-\log L = \sum_i \mathcal{L}_{\log}(y_i, z_i)$, the log loss





Log loss: −log of the probability of the true class



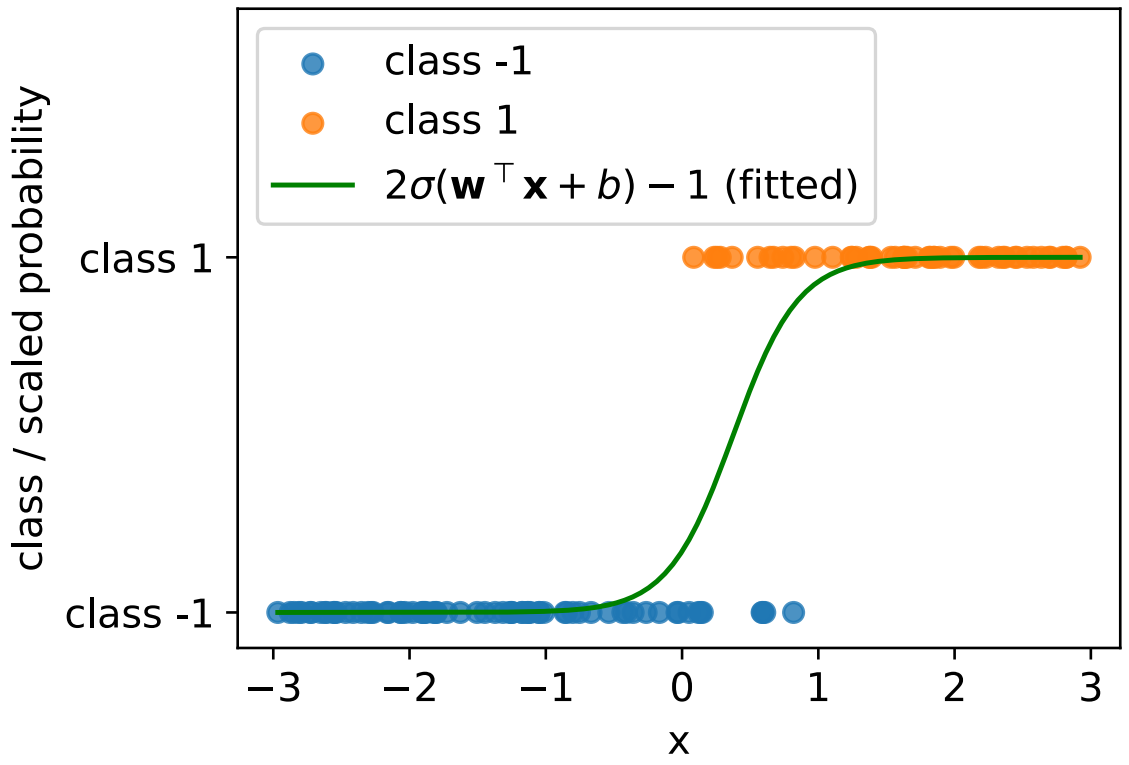
- $\mathcal{L} = -\log p_{\text{true}}$, where $p_{\text{true}} = \sigma(z)$ if $y = 1$ and $1 - \sigma(z)$ if $y = 0$
- **Far correct:** $p_{\text{true}} \to 1 \Rightarrow$ cost ≈ 0; **far wrong:** cost $\approx |z|$, linear
- The **cross-entropy** of classification networks (seventh lecture)

- **Gradient** of the summed loss ($y_i \in \{0, 1\}, \tilde{\mathbf{x}}_i = [1;\ \mathbf{x}_i]$):

$$\nabla_{\tilde{\mathbf{w}}} = \sum_{i=1}^{n} \left(\sigma(z_i) - y_i\right) \tilde{\mathbf{x}}_i$$

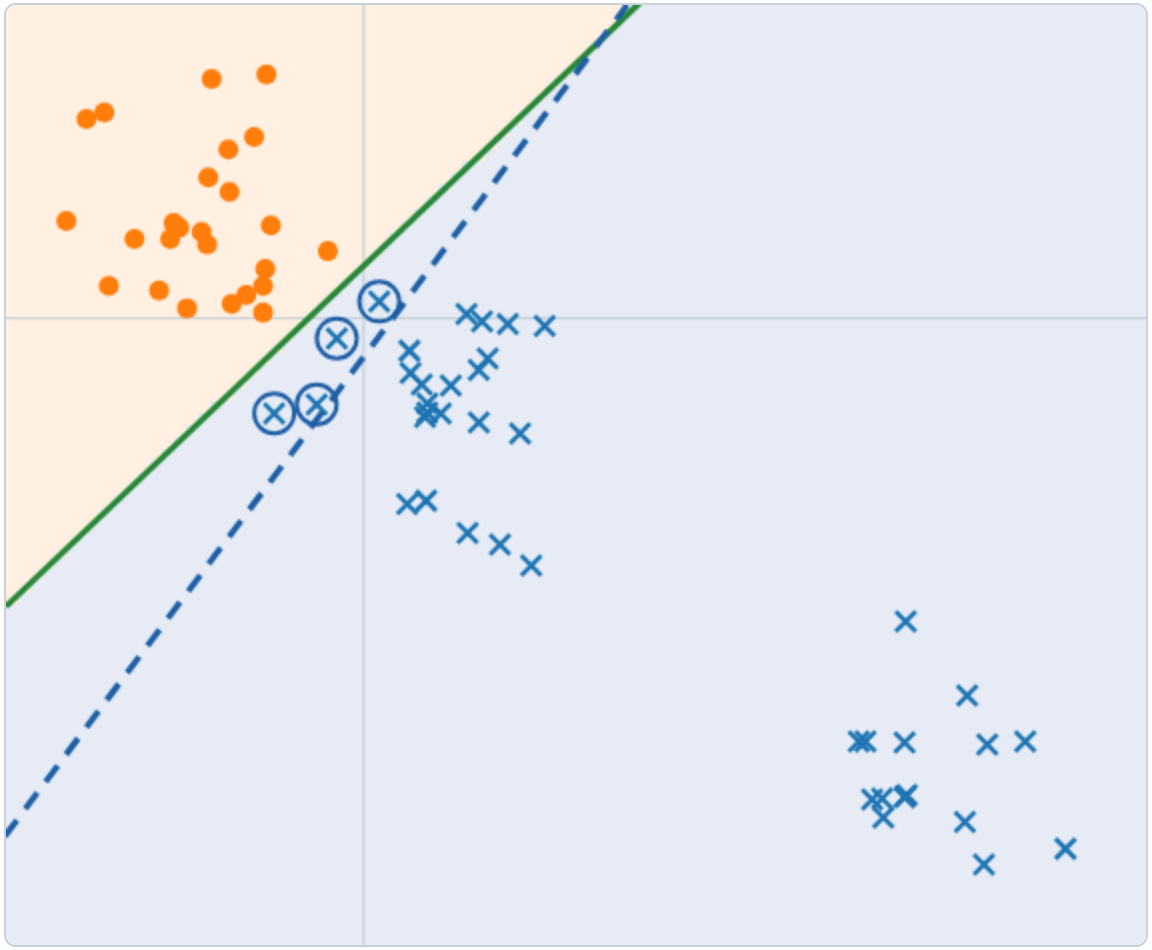
- σ inside $\Rightarrow$ **no closed form**; convex $\Rightarrow$ **gradient descent** (first lecture)

Logistic regression: behaviour



- **Calibrated probabilities** (under the correct model).
- **Convex** objective → gradient descent reaches the global optimum.
- **Less sensitive to outliers** than MSE: far, correct points cost ≈ 0.
- **Regularization** can be added.
- **Deep learning** uses this loss — a neural network replaces $\mathbf{w}^\top \mathbf{x}$ (seventh lecture).

Least squares vs. logistic: far points



- - - **least squares** (MSE classifier)
—— **logistic regression**

☑ far cluster (15 points of class +1, bottom right)

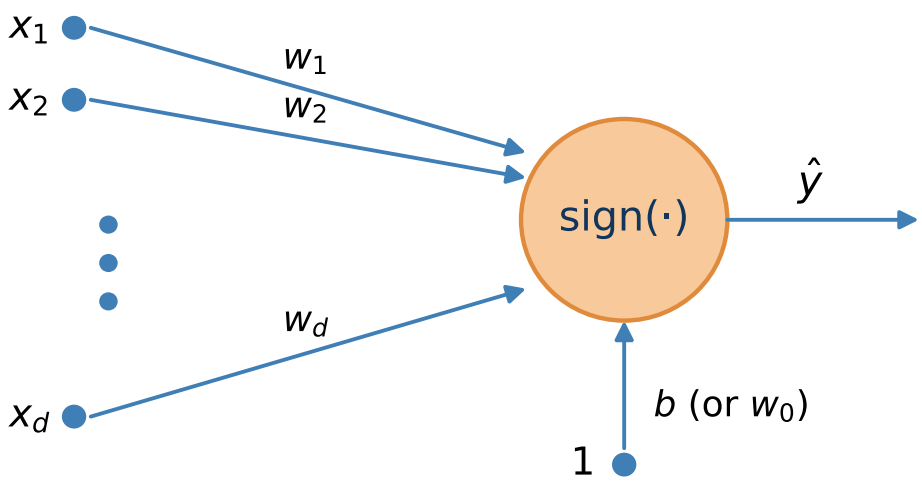
Add on click: ◉ +1 (blue ×) ○ −1 (orange ●)

Click to add a point (**shift-click** adds −1), **drag** a point to move it. reset

Misclassified out of 65: **least squares 4**, **logistic 0**.

Rings: misclassified by least squares (blue) or by logistic regression (green). The far points are all correctly on the +1 side.

Perceptron: model and loss



$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b), \qquad y \in \{-1, +1\}$$

$$\hat{y} = \text{sign}(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}) = \text{sign}\left(\sum_{j=0}^{d} w_j x_j\right)$$

bias trick: $x_0 = 1, w_0 = b$

Loss:

$$\mathcal{L}_{\text{perc}}(y, \tilde{\mathbf{w}}; \tilde{\mathbf{x}}) = \max\left(0,\ -y\, \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}\right)$$

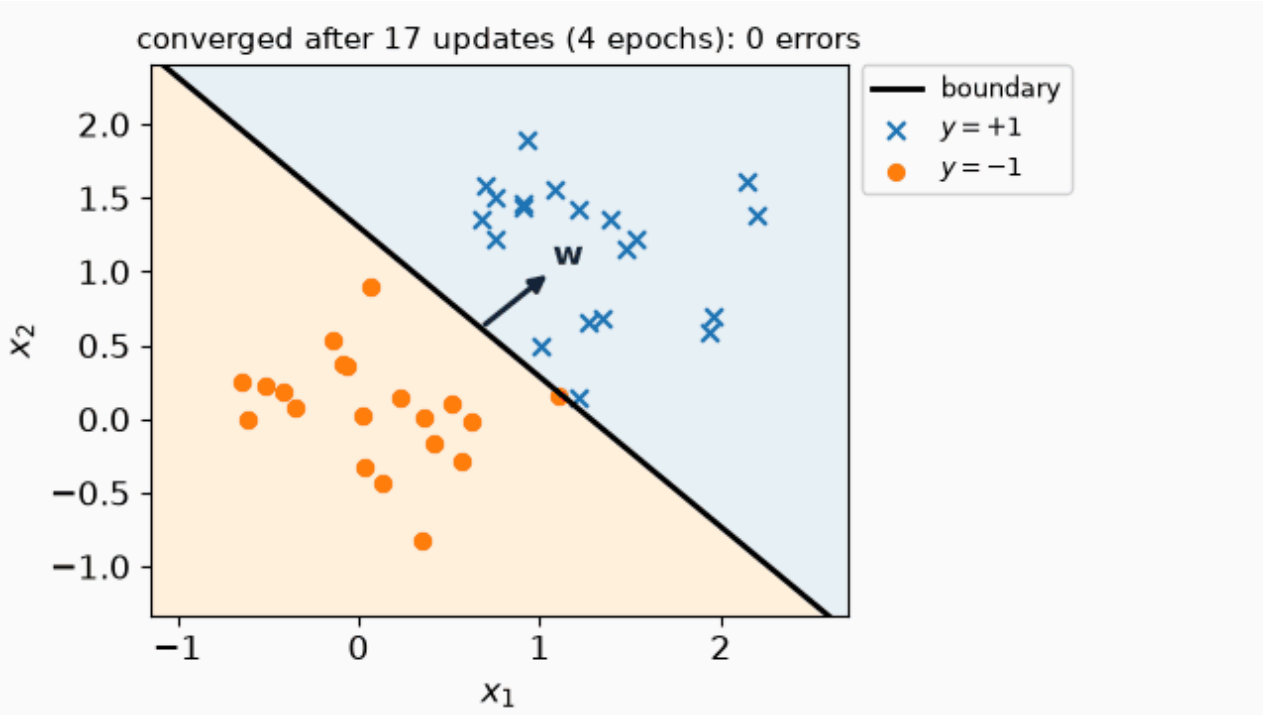
- **Correct side** ($y\, \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}} > 0$): loss 0 — nothing to learn.
- **Wrong side:** the loss grows linearly with $-y\, \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$.

Perceptron: learning rule

Loop over $i = 1, \dots, n$, repeat in epochs:

$$\text{if } y_i \, \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i \le 0 : \quad \tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta \, y_i \, \tilde{\mathbf{x}}_i$$

- **Misclassified** → shift $\tilde{\mathbf{w}}$ toward $y_i \tilde{\mathbf{x}}_i$; the boundary tilts toward the point.
- **Sign for free:** $y_i \in \{-1, +1\}$ flips the direction of the shift.
- **Learning rate** $\eta > 0$ only rescales $\tilde{\mathbf{w}}$ (from $\tilde{\mathbf{w}} = \mathbf{0}$).
- **Converges** provably only for linearly separable classes.
- **= one stochastic gradient step** on $\max(0, \, -y_i \, \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}_i)$ with step η — gradient descent (first lecture), one sample at a time.

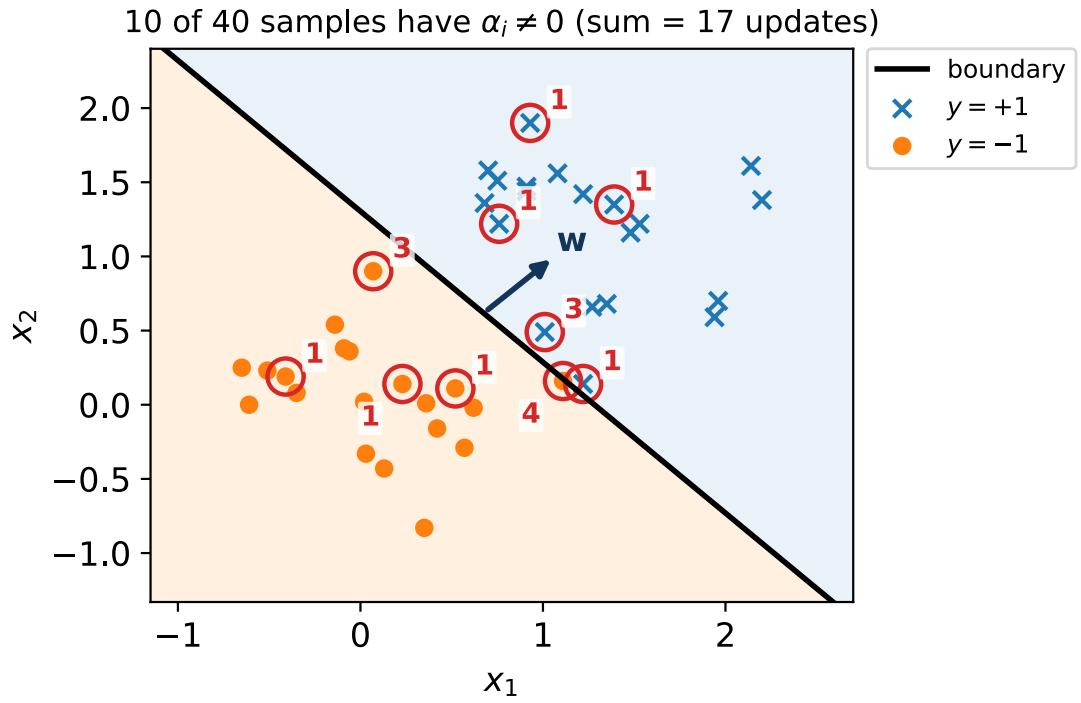


Step through it yourself — perceptron playground in the appendix

Perceptron algorithm: code


```python
def train_perceptron(X, y, epochs):
    n, d = X.shape
    w = np.zeros(d)
    for _ in range(epochs):
        for i in range(n):
            y_hat = X[i] @ w
            if y[i] * y_hat <= 0:
                w += y[i] * X[i]
    return w
```

```python
def predict_perceptron(X, w):
    return np.sign(X @ w)
```



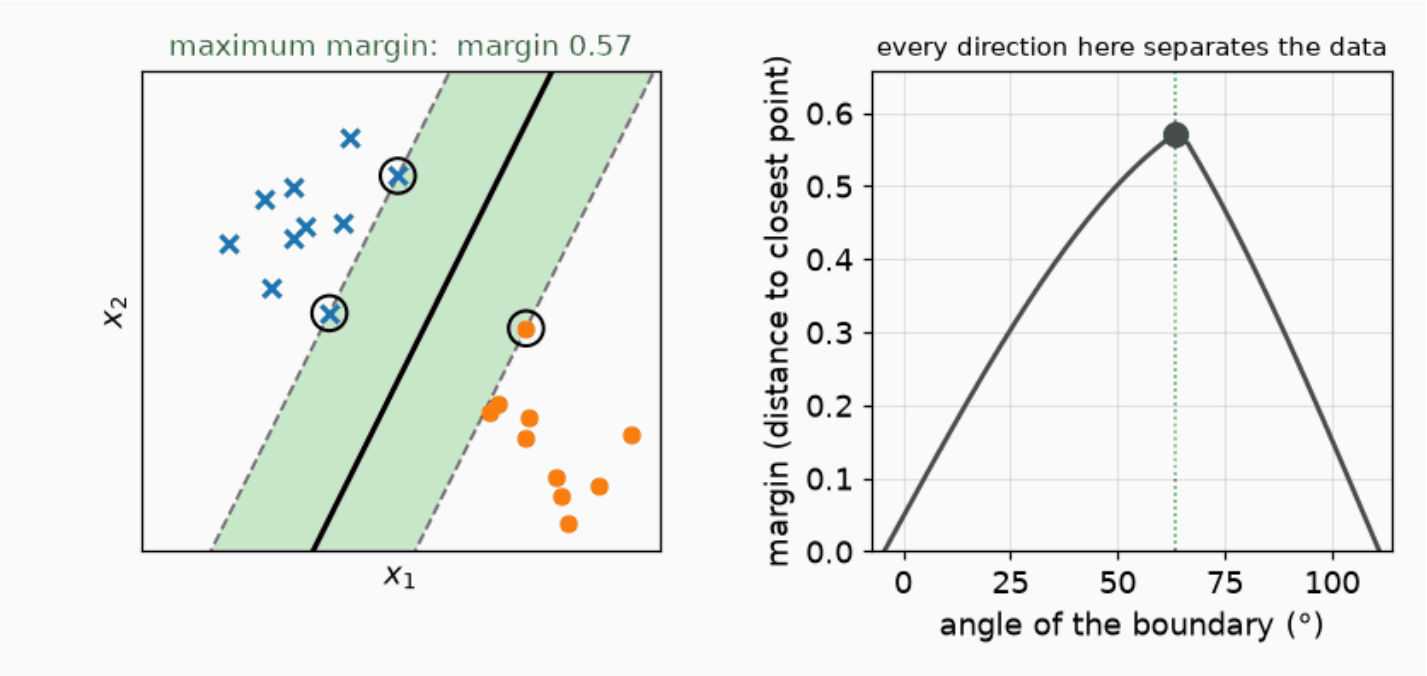
Key idea: $\mathbf{w}$ starts at $\mathbf{0}$ and every update adds $y_i \mathbf{x}_i \Rightarrow \mathbf{w} = \sum_{i=1}^{n} \alpha_i \, \mathbf{x}_i$, a weighted sum (linear combination) of the samples. These α_i are the weights of the **dual form** — an intuitive "proof" of it. runnable example

5 · Support vector machines

> If many boundaries are correct, take the one with the widest safety zone.

SVM intuition

- **Many** lines separate the data with **zero** training errors.
- Which one will classify **new** points best?
- **SVM:** the one with the largest **margin** — the widest empty band (“largest safe space”).
- Only the **closest points** (circled) decide where it lies.



From intuition to SVM optimization

1 · Distance of a point to the hyperplane $(\mathbf{w}, b)$: $\dfrac{|\,\mathbf{w}^\top \mathbf{x}_i + b\,|}{\|\mathbf{w}\|}$

2 · Fix the scale. $(\mathbf{w}, b)$ and $(2\mathbf{w}, 2b)$ describe the same line, so we may require

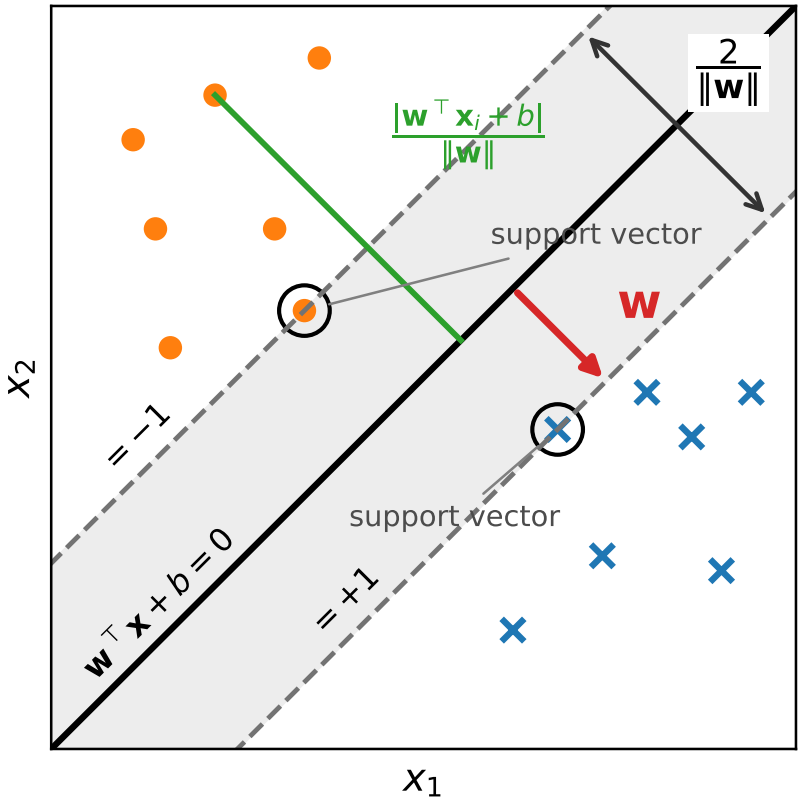
$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 \quad \text{for all } i,$$

with equality for the closest points (the **support vectors**) $\Rightarrow$ margin $= 1/\|\mathbf{w}\|$, band width $2/\|\mathbf{w}\|$.

3 · Maximize the margin. Maximizing $1/\|\mathbf{w}\|$ is the same as minimizing $\|\mathbf{w}\|^2$:

$$\boxed{\min_{\mathbf{w},b} \ \tfrac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \ i = 1, \ldots, n}$$

A convex quadratic problem (QP) $\Rightarrow$ one global optimum.



Hard-margin SVM: primal and dual

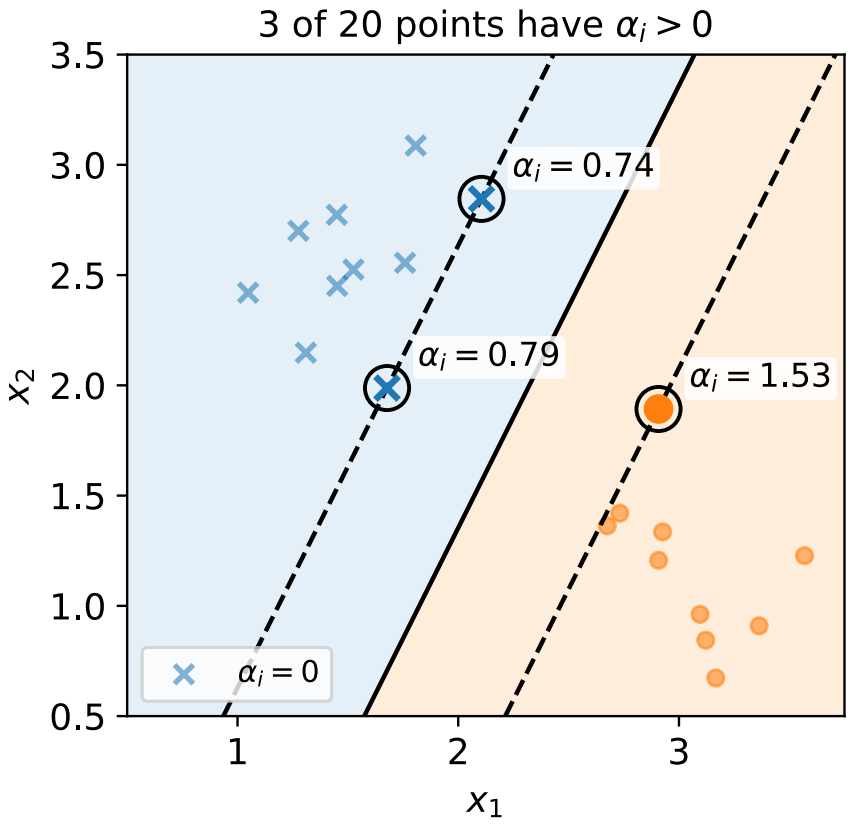
Primal — unknowns $\mathbf{w}, b$ (dimension $d+1$): the problem from the previous slide.

Dual — one unknown $\alpha_i \geq 0$ per sample (n unknowns):

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{n} \alpha_i - \tfrac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad \sum_{i=1}^{n} \alpha_i y_i = 0, \quad \boldsymbol{\alpha} \geq 0$$

$$\mathbf{w} = \sum_{i=1}^{n} \alpha_i y_i \mathbf{x}_i, \qquad \hat{y} = \operatorname{sign}\Big(\sum_{i \in \mathrm{SV}} \alpha_i y_i \, \mathbf{x}_i^\top \mathbf{x} + b \Big)$$

- The data enter **only through dot products** (red) → kernels.
- **Sparse:** $\alpha_i > 0$ only for points **on the margin**, $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$ — the **support vectors (SV)**. The other terms are zero → only the SVs are needed to get the same line, and prediction is fast.
- Both are convex **QP** problems: standard solvers, guaranteed global optimum.



code example · the dual by hand on two points — appendix

Soft-margin SVM

Real data **overlap** $\Rightarrow$ no line satisfies all hard constraints. **Slack** $\xi_i \geq 0$ lets point i violate its margin — at the price $C\xi_i$:

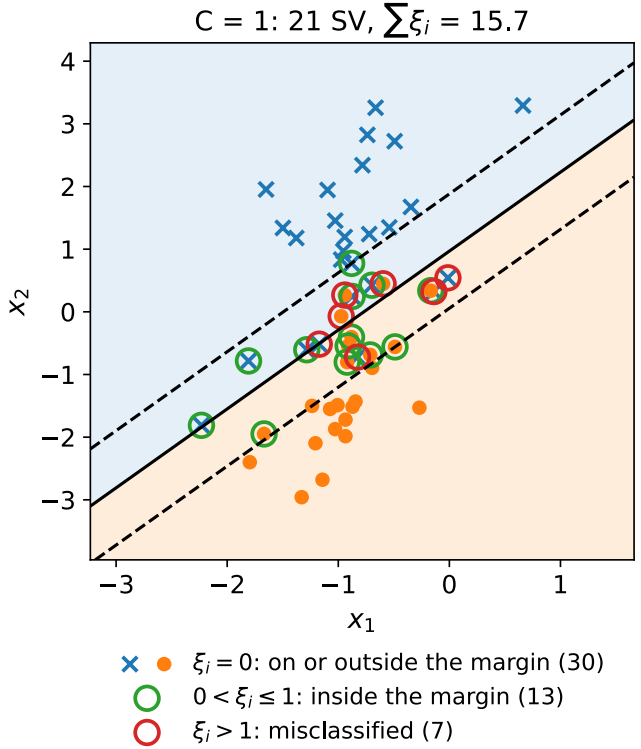
$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \tfrac{1}{2}\|\mathbf{w}\|^2 \ + \ C\sum_{i=1}^{n}\xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \ \ \xi_i \geq 0.$$

$C > 0$ trades off margin size vs. violations:

- $\xi_i = 0$: correct, **on or outside** the margin (30 points),
- $0 < \xi_i \leq 1$: correct but **inside** the margin (13),
- $\xi_i > 1$: **misclassified** (7).

Support vectors: every point with $\xi_i > 0$ plus those exactly on the edge ($20 + 1 = 21$ of 50 here); $\sum_i \xi_i = 15.7$.

- **Still a convex QP** → standard solver, global optimum.

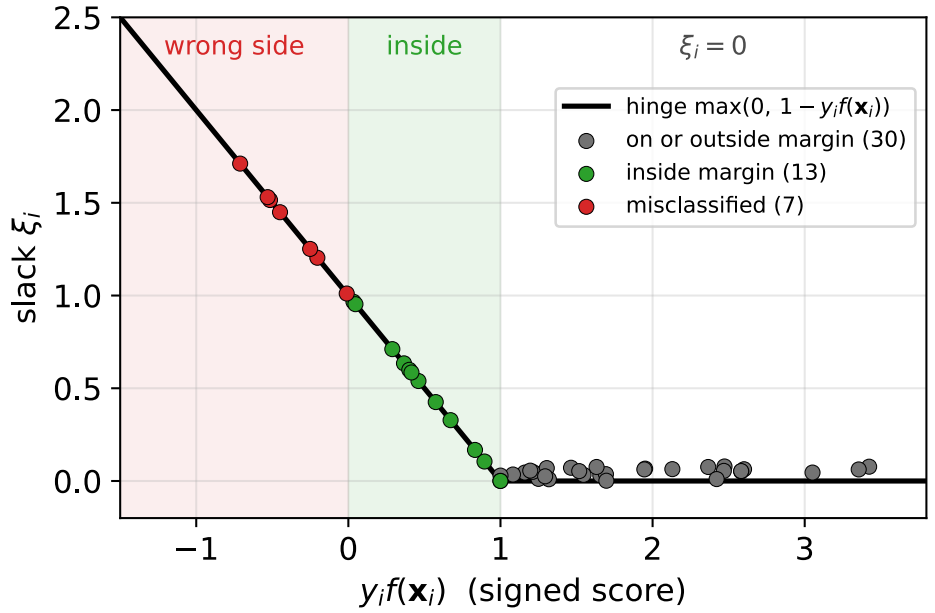


Soft-margin SVM = hinge loss + L2

At the optimum every slack is as small as the constraints allow, $\xi_i = \max\left(0,\ 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right)$. Plugging it back removes all constraints:

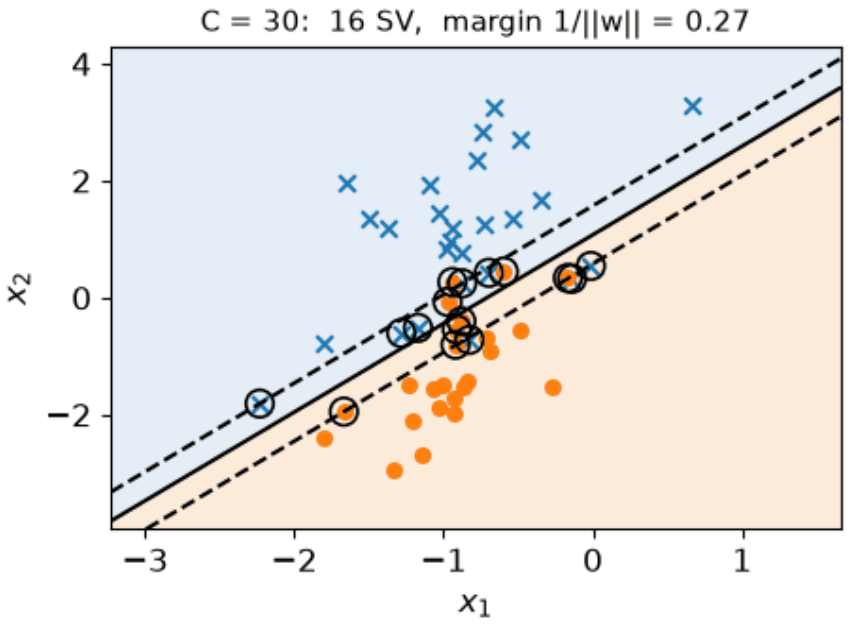
$$\min_{\mathbf{w},\, b} \underbrace{\tfrac{1}{2}\|\mathbf{w}\|^2}_{\text{margin (regularization)}} + \underbrace{C \sum_{i=1}^{n} \max\left(0,\ 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\right)}_{\text{hinge loss}}$$

- The **hinge loss** from the start of the lecture: the slack **is** the hinge.
- Score $y_i(\mathbf{w}^\top \mathbf{x}_i + b)$ $-0.5 \to \xi_i = 1.5$ (wrong side); $0.6 \to 0.4$ (inside); $2 \to 0$ (outside).
- Same form as every model today: **loss + regularization**.



The role of C

- $\frac{1}{2}\|\mathbf{w}\|^2$: prefers a **large margin** (smaller weights).
- Hinge loss: penalizes points **inside** the margin or **misclassified**.
- **Larger** C: fewer violations, tighter fit — risk of **overfitting**.
- **Smaller** C: wider margin, more tolerance to errors — risk of **underfitting**.
- C acts like $1/\lambda$ in ridge regression.
- Same data, C from 0.01 to 30: margin $2.54 \rightarrow 0.27$, support vectors $44 \rightarrow 16$.
- The hinge loss is also used e.g. for neural networks.



C-SVM vs. ν-SVM

C-SVM (the soft margin so far) · scikit-learn SVC

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i$$

$$\text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

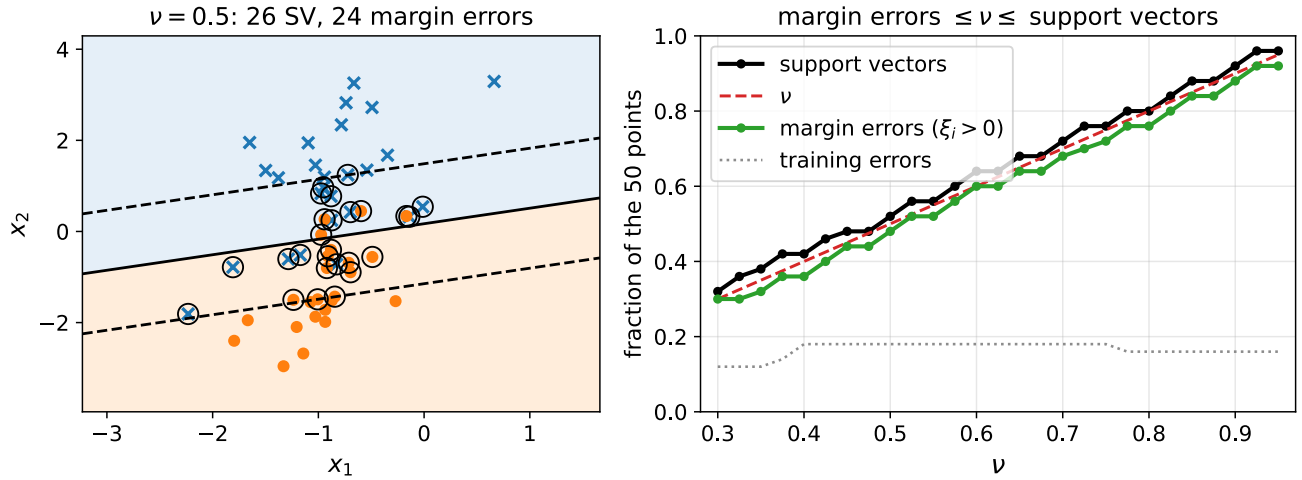
- $C > 0$, **unbounded**, scale depends on the data

ν-SVM · scikit-learn NuSVC

$$\min_{\mathbf{w},b,\boldsymbol{\xi},\rho} \ \tfrac{1}{2}\|\mathbf{w}\|^2 - \nu\rho + \tfrac{1}{n}\sum_{i=1}^{n}\xi_i$$

$$\text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq \rho - \xi_i, \quad \xi_i \geq 0, \quad \rho \geq 0$$

- $\nu \in (0, 1]$: **margin errors** ($\xi_i > 0$) $\leq \nu \leq$ **SVs** (as fractions)



Same family of solutions: every ν with $\rho > 0$ gives the boundary of some C (here $\nu = 0.5 \leftrightarrow C \approx 0.11$). Details: the ν-SVM appendix.

Kernel SVM

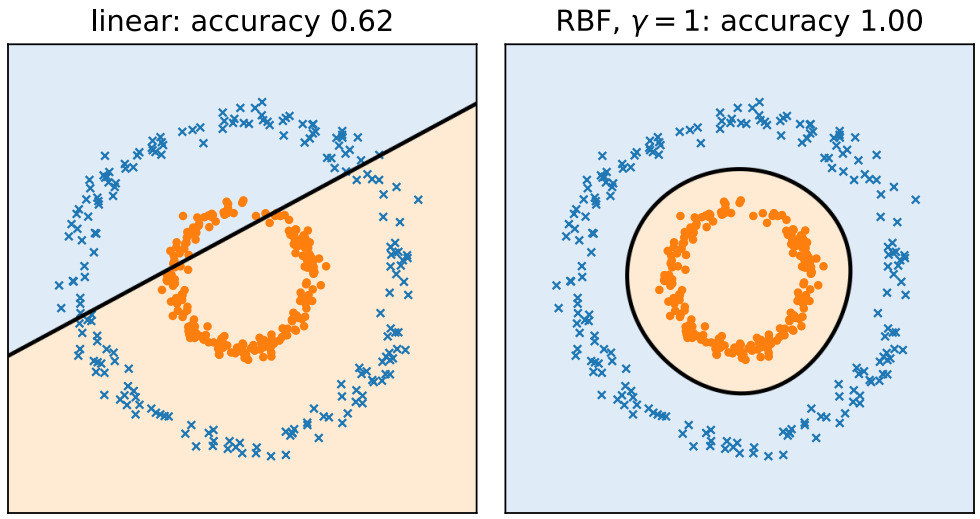
Soft-margin dual: the hard-margin dual plus one change — a box $0 \le \alpha_i \le C$. Now swap the dot product for a kernel, $\mathbf{x}_i^\top \mathbf{x}_j \;\to\; k(\mathbf{x}_i, \mathbf{x}_j)$:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{n} \alpha_i - \tfrac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \, k(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s.t.} \quad \sum_i \alpha_i y_i = 0, \;\; 0 \le \alpha_i \le C$$

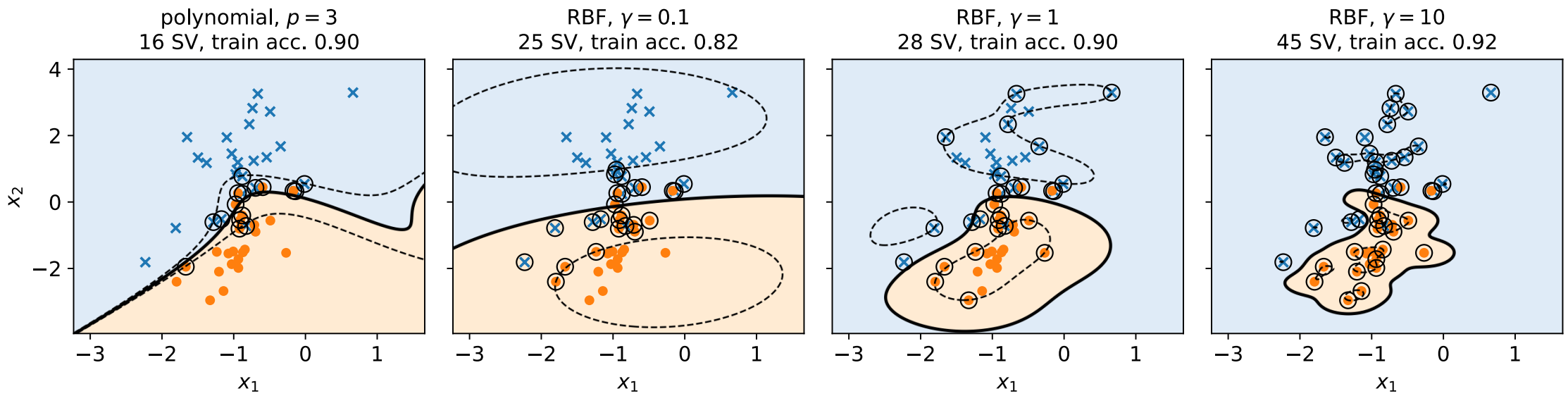
Score $f(\mathbf{x})$ (real-valued) and **decision** $\hat{y}$:

$$f(\mathbf{x}) = \sum_{i \in \mathrm{SV}} \alpha_i y_i \, k(\mathbf{x}_i, \mathbf{x}) \; + \; b, \qquad \hat{y} = \operatorname{sign}\big(f(\mathbf{x})\big)$$

- **Nonlinear** decision boundary in the input space, yet the **same** optimization problem.
- Its size depends on n (an $n \times n$ kernel matrix), **not** on the feature dimension — even an infinite one (RBF).
- The model is a list of support vectors with their α_i — no explicit $\mathbf{w}$.

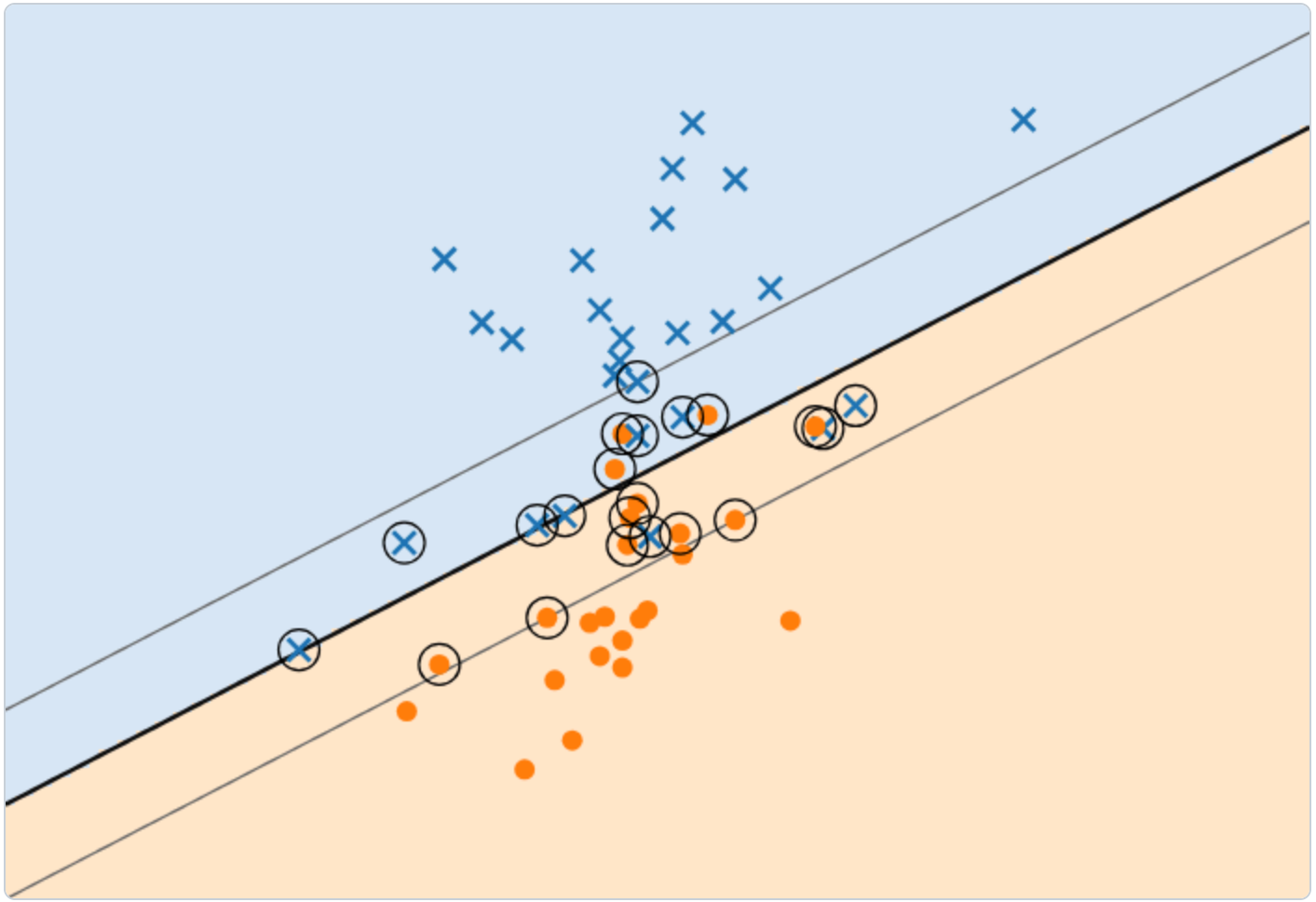


Kernel SVM: effect of the kernel and γ



- Same 50 points, same $C = 1$ — only the kernel changes.
- **Larger** γ (narrower RBF) $\Rightarrow$ more complex boundary, more support vectors; **higher** degree $p \Rightarrow$ more flexible boundary.
- $\gamma = 10$: 45 of 50 points are SVs — the model **memorizes** the noise.

SVM playground



Kernel: linear

C = 1

γ = 1

degree p = 3

Add on click: +1 (blue ×) −1 (orange ●)

Click to add a point (**shift-click** adds −1), **drag** a point to move it.

reset

21 support vectors of 50 · training accuracy **0.86** · margin 1/‖w‖ = **0.57**

The 50 points of the soft-margin slides. Thick line: boundary f = 0; thin grey lines: margin edges f = ±1; circles: support vectors. Solved in the browser by SMO (the libsvm algorithm).

SVM and kernel SVM in practice

Linear SVM

- $\mathbf{w}$: weights (importance) of features.
- $1/\|\mathbf{w}\|$ = margin (half of the band width $2/\|\mathbf{w}\|$).
- SVs: points on or inside the margin; they alone define $\mathbf{w}$, b.

Kernel SVM

- No explicit $\mathbf{w}$ in the input space.
- α_i: dual coefficients, nonzero only for SVs; larger α_i = more influence on the boundary.
- **Few** SVs $\Rightarrow$ sparser, simpler model; **many** SVs $\Rightarrow$ more complex boundary, higher memory cost.

Hyperparameters to choose

- C — trade-off between margin width and training errors (or ν in the ν-SVM).
- Kernel type and its parameters:
  - polynomial: degree p — larger = more complex model (overfitting),
  - RBF: $\gamma = 1/(2\sigma^2)$ (inverse width) — larger = more complex model.
- Tune them on a (logarithmic) grid with cross-validation.
- **Standardize** the features first.

code example — application

6 · Recap

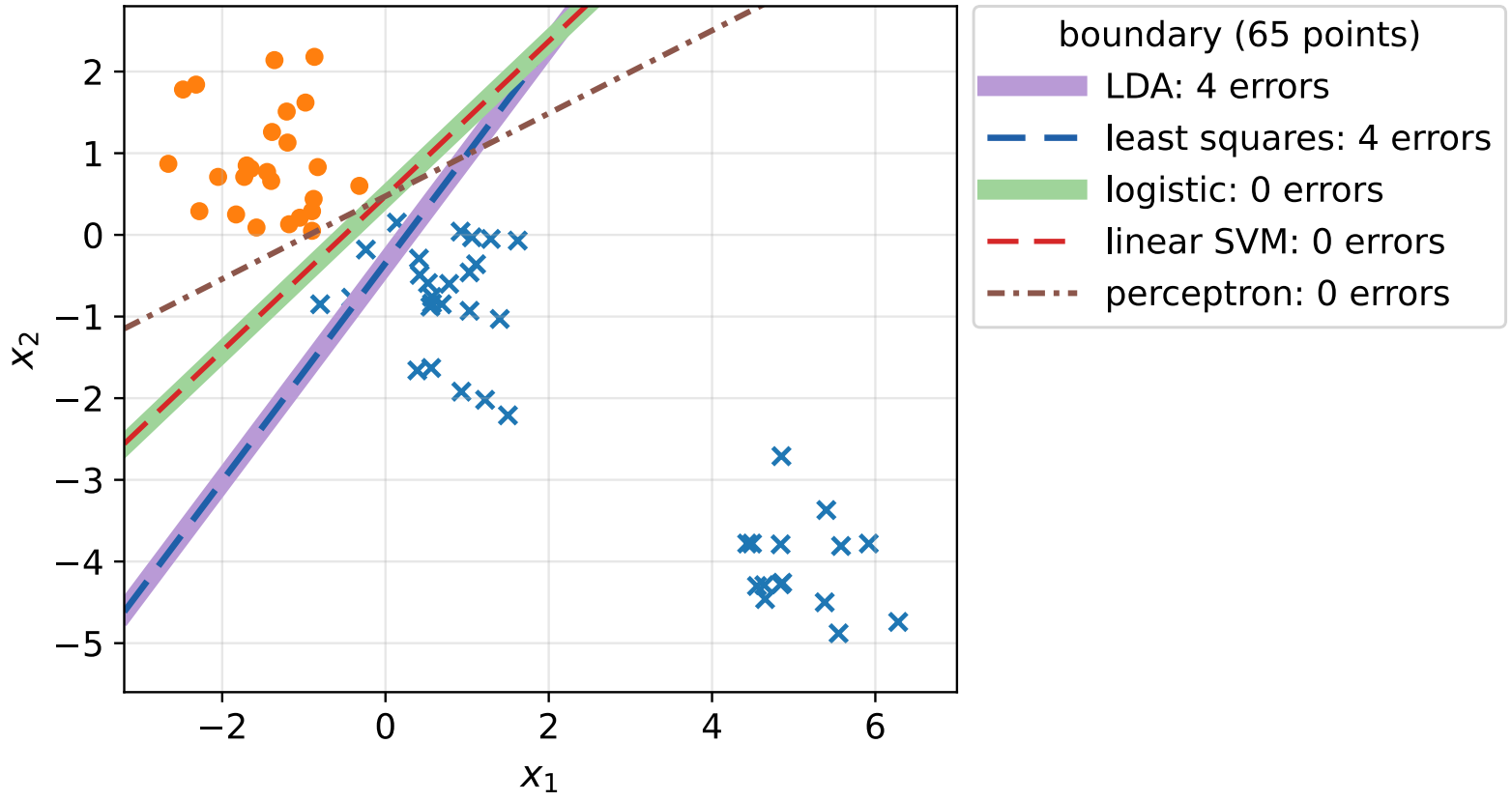
> The whole chain from OLS to kernel SVM on one page and one picture — then three sets of questions to check that it holds together.

Summary: one model, many losses

Method	Loss $\mathcal{L}(y, z)$	Regularizer	Closed form?	Primal / dual / kernel
OLS	$(y-z)^2$	—	✓ $(X^\top X)^{-1} X^\top \mathbf{y}$	primal
Ridge	$(y-z)^2$	$\lambda \lVert \mathbf{w} \rVert_2^2$	✓ $(X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$	primal or dual
LASSO	$(y-z)^2$	$\lambda \lVert \mathbf{w} \rVert_1$	✗ convex solver	primal
LAD	$\lvert y-z \rvert$	—	✗ linear program	primal
Kernel ridge	$(y-z)^2$	$\lambda \lVert \mathbf{w} \rVert_2^2$	✓ $(K + \lambda I)^{-1} \mathbf{y}$	kernel (dual)
LDA	Fisher criterion, no per-sample loss	—	✓ $S_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$	primal
MSE classifier	$(1-yz)^2$	—	✓ as OLS	primal
Logistic	$\log(1 + e^{-yz})$	optional L_2 / L_1	✗ gradient descent	primal
Perceptron	$\max(0, -yz)$	—	✗ update rule	primal; $\mathbf{w} = \sum_i \alpha_i \mathbf{x}_i$
SVM	$\max(0, 1-yz)$	$\frac{1}{2} \lVert \mathbf{w} \rVert^2$ ($C \sim 1/\lambda$)	✗ QP	primal or dual
Kernel SVM	$\max(0, 1-yz)$	$\frac{1}{2} \lVert \mathbf{w} \rVert^2$	✗ QP	kernel (dual)

Score $z = \mathbf{w}^\top \mathbf{x} + b$; regression losses use the residual $y - z$, classification losses the margin yz with $y \in \{-1, +1\}$.

Summary: five linear classifiers on one dataset



Same model $\mathrm{sign}(\mathbf{w}^\top \mathbf{x} + b)$, five losses → **different** lines. LDA and least squares are pulled by the far, correct points; losses that stop at the correct side are not.

Recap quiz I — linear models basics

- Explain the **bias trick**. How do we rewrite $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$ using $\tilde{\mathbf{x}}$ and $\tilde{\mathbf{w}}$? Why is this practical?
- What is the difference between **regression** and **classification** for linear models? State the role of the decision boundary $\mathbf{w}^\top \mathbf{x} + b = 0$.
- Why is **MSE** sensitive to outliers while **MAE** is more robust? Give the intuition.
- Write the generic **empirical risk minimization** form: average loss + regularization. What are typical choices of $\Omega(\mathbf{w})$ and their effects?
- Compare **logistic regression** with the **MSE classifier**.

Recap quiz II — OLS, regularization, polynomials

- Derive the **closed-form solution for OLS** from the objective $g(\mathbf{w}) = ||X\mathbf{w} - \mathbf{y}||_2^2$ (full steps in the appendix).
- Contrast **Ridge** (L_2) vs. **LASSO** (L_1): coefficient shrinkage, feature selection, and the existence of a closed form.
- How do you build **polynomial regression** via feature expansion? Explain why it remains linear in the parameters.
- Using a simple example, explain how the **polynomial degree** and **regularization** jointly shape the fit.

Recap quiz III — perceptron, SVM, kernels

- Explain how the **perceptron** update implies that $\mathbf{w}$ is a linear combination of the training samples: $\mathbf{w} = \sum_i \alpha_i \mathbf{x}_i$.
- Define the **margin** in SVM and explain why maximizing the margin leads to minimizing $\frac{1}{2}\|\mathbf{w}\|^2$.
- Contrast **hard-margin** vs. **soft-margin** SVM. What is the role of C and what do the **slack** variables ξ_i represent? Why is the soft-margin SVM the same as **hinge loss + L2**?
- Interpret **support vectors** and the dual coefficients α_i. What does *many* vs. *few* SVs mean for the model?
- Write the SVM **dual objective** and indicate where **dot products** (or kernels) appear between samples.
- Explain the **kernel trick**.
- Which **hyperparameters** do we tune in kernel SVMs (poly vs. RBF), and how do they affect the complexity of the decision boundary?
- Why is the dual/kernel approach beneficial when the feature dimension d is very large or infinite?

Appendix: Linear regression

> Optional steps behind the regression slides — the main lecture needs only the results.

OLS: step-by-step derivation

Expand the squared norm:

$$g(\mathbf{w}) = (X\mathbf{w} - \mathbf{y})^\top (X\mathbf{w} - \mathbf{y}) = \mathbf{w}^\top X^\top X\,\mathbf{w} - 2\,\mathbf{w}^\top X^\top \mathbf{y} + \mathbf{y}^\top \mathbf{y}$$

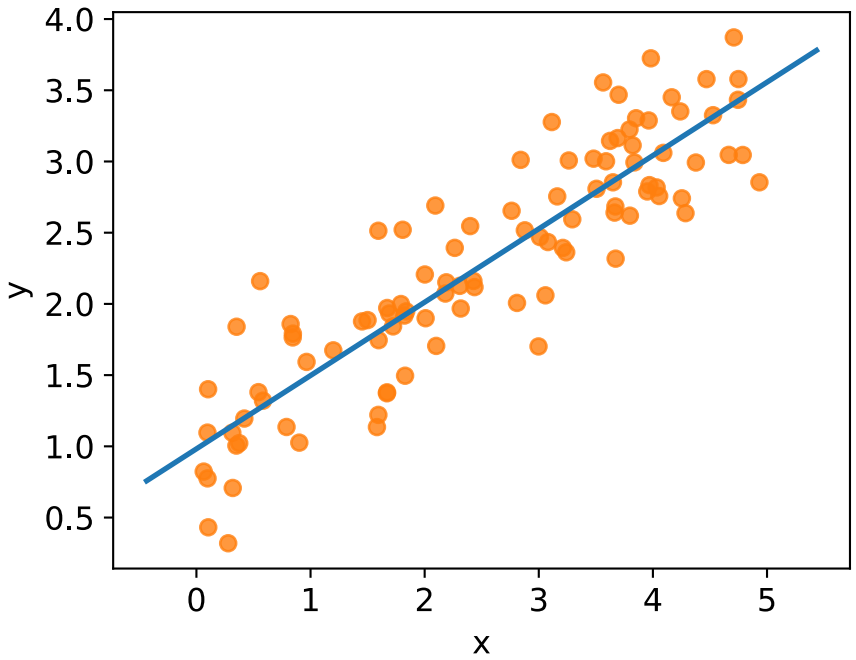
Differentiate with $\nabla_{\mathbf{w}}(\mathbf{w}^\top A\mathbf{w}) = 2A\mathbf{w}$ (A symmetric) and $\nabla_{\mathbf{w}}(\mathbf{w}^\top \mathbf{c}) = \mathbf{c}$:

$$\nabla g(\mathbf{w}) = 2X^\top X\,\mathbf{w} - 2X^\top \mathbf{y} = 2X^\top (X\mathbf{w} - \mathbf{y})$$

Set to zero → the *normal equations*:

$$X^\top X\,\mathbf{w} = X^\top \mathbf{y} \quad \Rightarrow \quad \mathbf{w}^* = (X^\top X)^{-1} X^\top \mathbf{y} = X^\dagger \mathbf{y}$$

- $X^\top X$ is $d \times d$ (with the bias column $(d{+}1) \times (d{+}1)$) — its size depends on the number of **features**, not of samples
- invertible when the columns of X are linearly independent; otherwise `np.linalg.pinv` still returns a least-squares solution (the one with the smallest $\|\mathbf{w}\|$)



The same formula for $n = 100$ points.

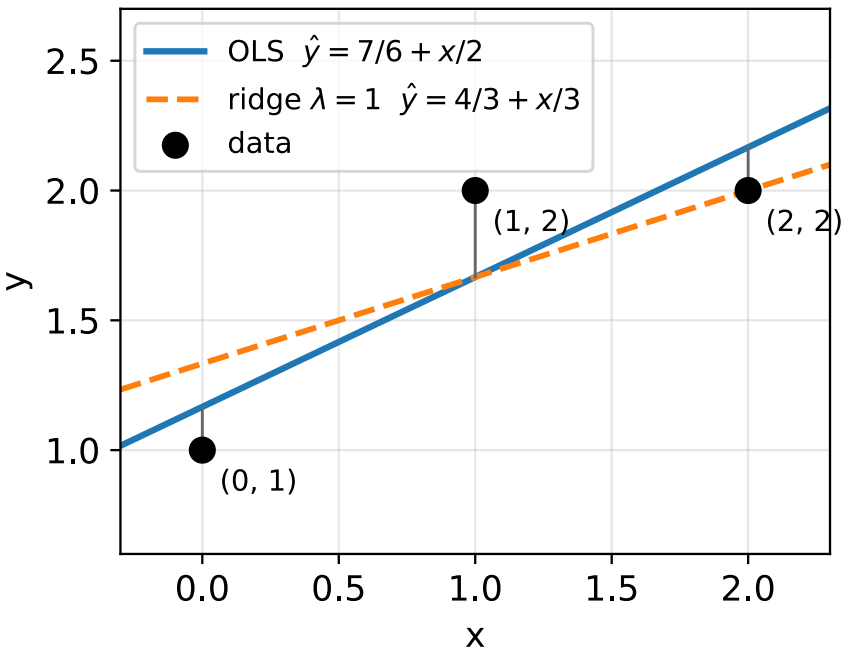
Ridge by hand: the same three points

Data $(0,1),(1,2),(2,2)$: $X^\top X = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}$, $X^\top \mathbf{y} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$, OLS $\tilde{\mathbf{w}} = \begin{bmatrix} 7/6 \\ 1/2 \end{bmatrix}$.

Ridge with $\lambda = 1$ on the slope only (the intercept is usually not penalized):

$$\tilde{\mathbf{w}} = \left(X^\top X + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \right)^{-1} X^\top \mathbf{y} = \tfrac{1}{9} \begin{bmatrix} 6 & -3 \\ -3 & 3 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 4/3 \\ 1/3 \end{bmatrix}$$

The slope shrinks from $\frac{1}{2}$ to $\frac{1}{3}$; the intercept moves to keep the line near the data.



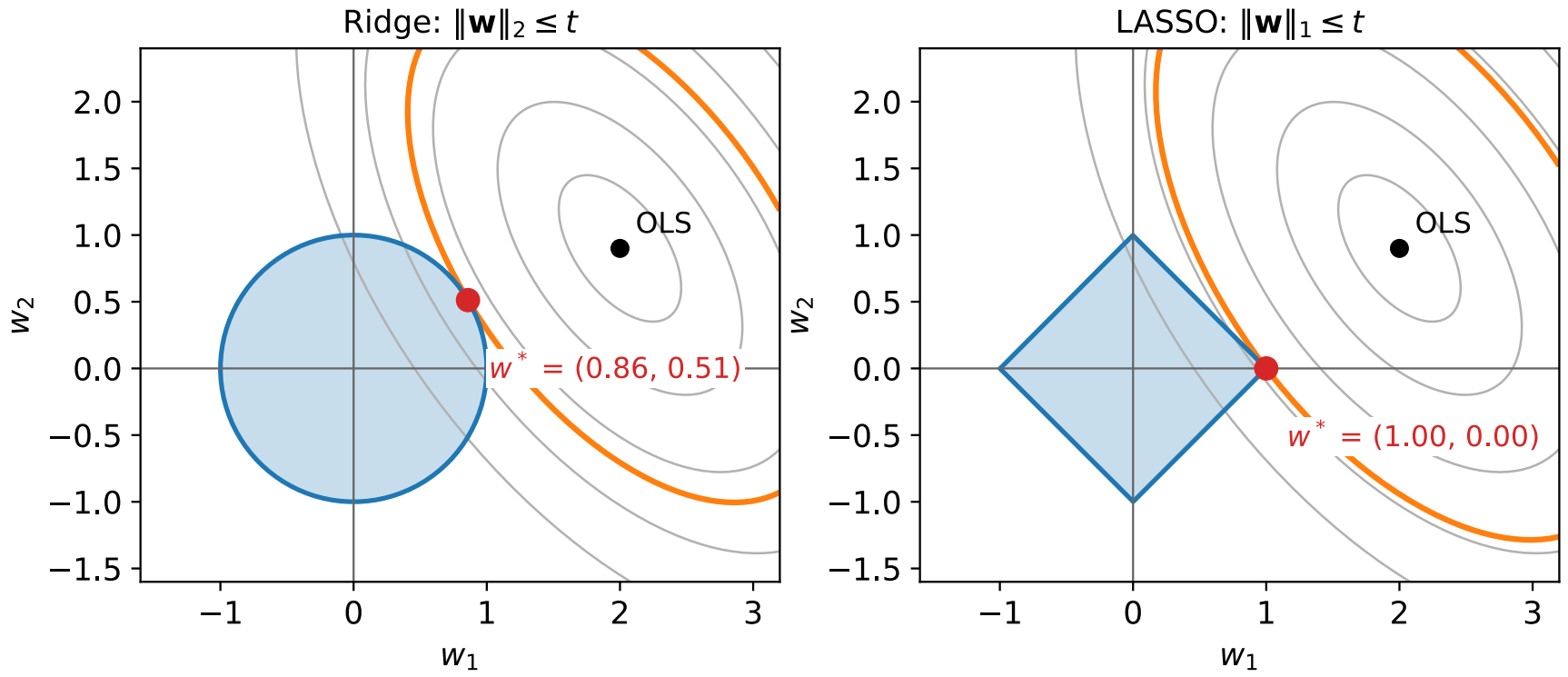
LAD as a linear program

Replace every absolute value by an auxiliary variable $t_i \geq |\mathbf{x}_i^\top \mathbf{w} - y_i|$:

$$\underset{\mathbf{w},\ \mathbf{t}}{\text{minimize}} \ \sum_{i=1}^{n} t_i \qquad \text{s.t.} \qquad -t_i \ \leq \ \mathbf{x}_i^\top \mathbf{w} - y_i \ \leq \ t_i, \quad i = 1, \ldots, n$$

- **Linear** objective and **linear** constraints → a linear program ($d + n$ variables, $2n$ inequalities)
- at the optimum every t_i is pushed down to exactly $|\mathbf{x}_i^\top \mathbf{w} - y_i|$, so the minimum is the L_1 error
- the optimum lies at a vertex of the feasible set — for a line, some optimal LAD fit passes **through two data points**
- this is how the blue line of the MSE-vs-MAE figure is computed (`scipy.optimize.linprog`)

Why LASSO gives exact zeros



- The penalized form equals a **constrained** one: $\min \|X\mathbf{w} - \mathbf{y}\|_2^2$ s.t. $\Omega(\mathbf{w}) \le t$ (each λ has its t)
- The solution is where the growing loss ellipse **first touches** the allowed region
- The L_1 diamond has **corners on the axes** → the touch is often at a corner → some $w_j = 0$; the L_2 disc has no corners

Appendix: Nonlinear features and kernels

> Optional algebra behind the kernel trick and kernel ridge regression.

Kernel trick: the algebra

1. Explicit feature expansion: $\Phi(\mathbf{x}) = (x_1^2,\ x_2^2,\ \sqrt{2}\,x_1 x_2)$

$$\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j) = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2}$$

2. Direct computation (kernel trick):

$$(\mathbf{x}_i^\top \mathbf{x}_j)^2 = (x_{i1}x_{j1} + x_{i2}x_{j2})^2 = x_{i1}^2 x_{j1}^2 + x_{i2}^2 x_{j2}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2}$$

Conclusion: computing $\Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$ explicitly is the same as computing $(\mathbf{x}_i^\top \mathbf{x}_j)^2$ — for every pair, not only $(1, 2)$ and $(3, 1)$. $\Rightarrow$ the kernel function avoids the explicit high-dimensional mapping.

Kernel matrix (Gram matrix) for n samples $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\}$:

$$K = \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \ldots & k(\mathbf{x}_1, \mathbf{x}_n) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \ldots & k(\mathbf{x}_2, \mathbf{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & k(\mathbf{x}_n, \mathbf{x}_2) & \ldots & k(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix}$$

With the linear kernel $K = XX^\top$. A valid kernel gives a symmetric, positive semidefinite K for any set of samples.

Dual ridge regression: derivation

Start from the primal solution: $(X^\top X + \lambda I)\,\mathbf{w} = X^\top \mathbf{y}$

1. **Rearrange** ($\lambda > 0$):

$$\lambda \mathbf{w} = X^\top(\mathbf{y} - X\mathbf{w}) \;\Rightarrow\; \mathbf{w} = X^\top \boldsymbol{\alpha}, \qquad \boldsymbol{\alpha} = \tfrac{1}{\lambda}(\mathbf{y} - X\mathbf{w})$$

2. **Substitute** $\mathbf{w} = X^\top \boldsymbol{\alpha}$ into the definition of $\boldsymbol{\alpha}$:

$$\lambda \boldsymbol{\alpha} = \mathbf{y} - XX^\top \boldsymbol{\alpha} \;\Rightarrow\; (XX^\top + \lambda I)\,\boldsymbol{\alpha} = \mathbf{y}$$

3. **Dual objective** — its gradient is zero exactly there:

$$\underset{\boldsymbol{\alpha}}{\text{minimize}} \;\; \tfrac{1}{2}\,\boldsymbol{\alpha}^\top (XX^\top + \lambda I)\boldsymbol{\alpha} - \mathbf{y}^\top \boldsymbol{\alpha} \qquad \nabla = (XX^\top + \lambda I)\boldsymbol{\alpha} - \mathbf{y} = \mathbf{0}$$

> α_i = **residual of sample i divided by λ**; the weights are a combination of the training samples.

Primal, dual and kernel ridge: all formulas

	Primal	Dual	Kernel (KRR)
objective	$\|X\mathbf{w}-\mathbf{y}\|_2^2+\lambda\|\mathbf{w}\|_2^2$	$\frac{1}{2}\boldsymbol{\alpha}^\top(XX^\top+\lambda I)\boldsymbol{\alpha}-\mathbf{y}^\top\boldsymbol{\alpha}$	$\frac{1}{2}\boldsymbol{\alpha}^\top(K+\lambda I)\boldsymbol{\alpha}-\mathbf{y}^\top\boldsymbol{\alpha}$
solution	$\mathbf{w}^*=(X^\top X+\lambda I)^{-1}X^\top\mathbf{y}$	$\boldsymbol{\alpha}^*=(XX^\top+\lambda I)^{-1}\mathbf{y}$	$\boldsymbol{\alpha}^*=(K+\lambda I)^{-1}\mathbf{y}$
matrix inverted	$d\times d$	$n\times n$	$n\times n$
one new sample	$\mathbf{x}_{\text{new}}^\top\mathbf{w}$	$\sum_i \alpha_i\,\mathbf{x}_i^\top\mathbf{x}_{\text{new}}=\boldsymbol{\alpha}^\top X\,\mathbf{x}_{\text{new}}$	$\sum_i \alpha_i\,k(\mathbf{x}_i,\mathbf{x}_{\text{new}})$
many new samples	$X_{\text{new}}\,\mathbf{w}$	$X_{\text{new}}X^\top\boldsymbol{\alpha}$	$K_{\text{new}}\,\boldsymbol{\alpha}$

Code example · $X_{\text{new}}\in\mathbb{R}^{m\times d}$ holds m new samples as rows; $(K_{\text{new}})_{ij}=k(\mathbf{x}_{\text{new},i},\mathbf{x}_j)$ is $m\times n$. Link between primal and dual: $\mathbf{w}=X^\top\boldsymbol{\alpha}=\sum_i \alpha_i\mathbf{x}_i$.

Appendix: Linear classification

> The LDA threshold with priors, the logistic gradient by hand, and a perceptron to step through.

LDA classifier: Fisher criterion and threshold

Fisher criterion (maximize separation / penalize spread):

$$J(\mathbf{w}) = \frac{(\tilde{\mu}_1 - \tilde{\mu}_2)^2}{\tilde{s}_1^{\,2} + \tilde{s}_2^{\,2}} = \frac{\big(\mathbf{w}^\top(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)\big)^2}{\mathbf{w}^\top S_W \mathbf{w}}, \qquad S_W = S_1 + S_2,$$

maximized by $\mathbf{w} = S_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$ — any nonzero multiple gives the same direction.

Threshold with priors π_1, π_2 (Gaussian classes with a shared covariance). The scale of $\mathbf{w}$ now matters: use the pooled covariance $\Sigma_W = S_W/(n-2)$,

$$\boxed{\mathbf{w} = \Sigma_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)} \qquad \text{class 1 if } \mathbf{w}^\top \mathbf{x} > t^*, \qquad \boxed{t^* = \tfrac{1}{2}(\boldsymbol{\mu}_1 + \boldsymbol{\mu}_2)^\top \mathbf{w} \;-\; \ln \frac{\pi_1}{\pi_2}}$$

- **Equal priors:** $\ln 1 = 0$ → the midpoint of the projected means (the figure: 90 + 90 points).
- **Class 1 more frequent** ($\pi_1 > \pi_2$): t^* drops, so class 1 is predicted more often.
- **With** $\mathbf{w} = S_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)$ the prior term becomes $\ln(\pi_1/\pi_2)/(n-2)$.

Logistic regression: one gradient step by hand

One loss, two label codings. Since $\sigma(-z) = 1 - \sigma(z)$:

$$-\log \sigma(z) = \log\left(1 + e^{-z}\right), \qquad -\log\left(1 - \sigma(z)\right) = \log\left(1 + e^{z}\right),$$

so $y = 1 \leftrightarrow +1$ and $y = 0 \leftrightarrow -1$ both give $\log(1 + e^{-yz})$.

Gradient for one sample ($y \in \{0, 1\}, z = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$; the main slide sums it over the samples):

$$\frac{\partial \mathcal{L}_{\log}}{\partial \tilde{\mathbf{w}}} = \big(\sigma(z) - y\big)\, \tilde{\mathbf{x}}$$

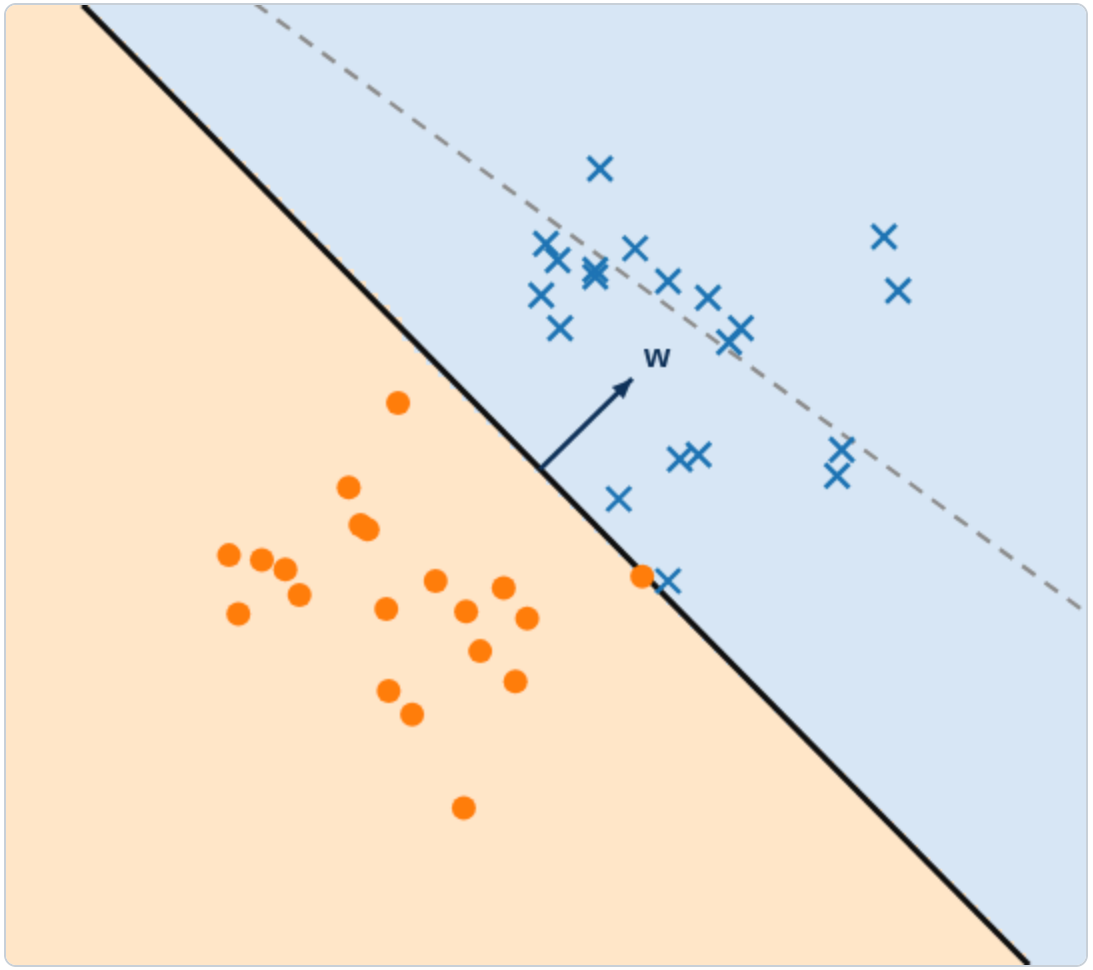
one gradient-descent step: $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} - \eta\,(\sigma(z) - y)\,\tilde{\mathbf{x}}$.

One sample: $x = 2, y = 1 \Rightarrow \tilde{\mathbf{x}} = (1,\ 2)$; start at $\tilde{\mathbf{w}} = (b,\ w) = (0,\ 0), \eta = 1$.

1. $z = 0,\ \sigma(z) = 0.5,\ \ \text{loss} = \ln 2 \approx 0.69$
2. gradient $= (0.5 - 1)\,(1,\ 2) = (-0.5,\ -1)$
3. $\tilde{\mathbf{w}} \leftarrow (0,\ 0) - (-0.5,\ -1) = (0.5,\ 1)$
4. $z = 0.5 \cdot 1 + 1 \cdot 2 = 2.5,\ \sigma(z) \approx 0.92,\ \ \text{loss} \approx 0.08$

One step moves the probability of the true class from 0.5 to 0.92.

Perceptron playground



step | one epoch | run | restart from w = 0 | reset data

Add on click: ◉ +1 (blue ×) ○ −1 (orange ●)
Click to add a point (**shift-click** adds −1), **drag** a point to move it. Changing the data restarts from **w** = 0.

Converged: 17 updates, the last in epoch 4; epoch 5 passes with no update.
$\tilde{w} = (b, w_1, w_2) = (-3.00, 2.34, 2.30)$
Updates: **17**, errors now: **0** of 40.

Black line: the current boundary with its normal **w**; grey dashed: the boundary before the last update; the ring marks the sample just checked (red = update, green = already correct). The 40 points are those of the animation.

Appendix: Support vector machines

> Optional detail: the dual solved by hand, why ν bounds the margin errors and the support vectors, and six ν-SVM figures.

Hard-margin SVM by hand: two points

$\mathbf{x}_1 = (1,1), y_1 = +1$ and $\mathbf{x}_2 = (-1,-1), y_2 = -1$.

Geometry: the boundary is the perpendicular bisector $x_1 + x_2 = 0$; each point is at distance $\sqrt{2}$ from it, so the margin is $1/\|\mathbf{w}\| = \sqrt{2}$ and $\|\mathbf{w}\| = 1/\sqrt{2}$.

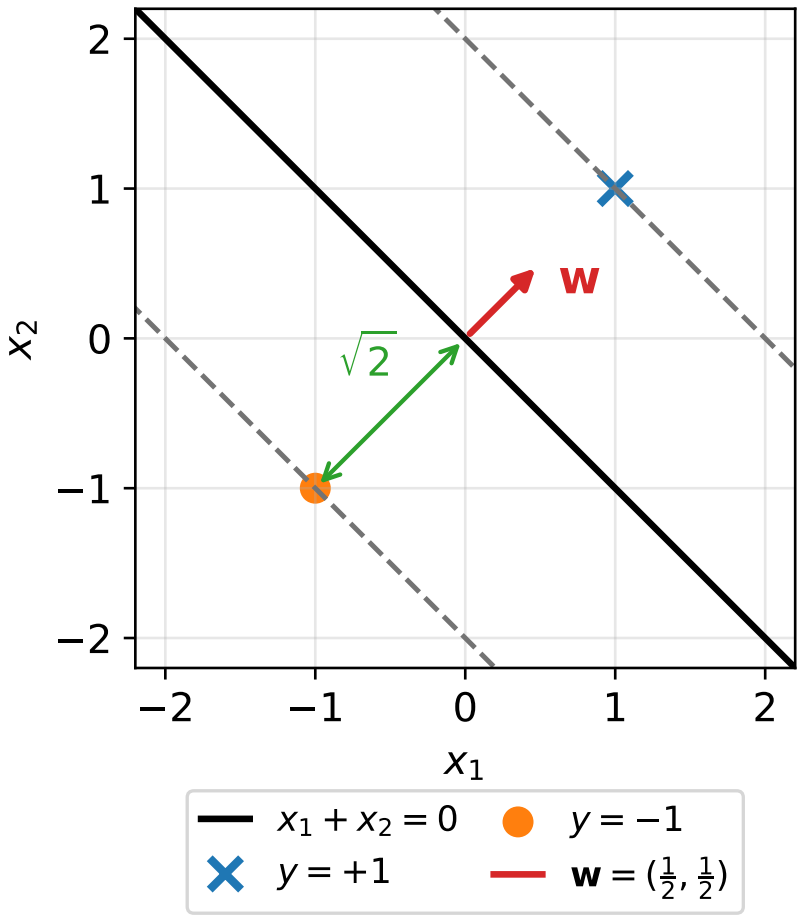
Primal: $\mathbf{w} = (\frac{1}{2}, \frac{1}{2}), b = 0$ — check $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$ for both points; objective $\frac{1}{2}\|\mathbf{w}\|^2 = \frac{1}{4}$.

Dual: $\mathbf{x}_1^\top \mathbf{x}_1 = \mathbf{x}_2^\top \mathbf{x}_2 = 2, \mathbf{x}_1^\top \mathbf{x}_2 = -2$; the constraint $\alpha_1 y_1 + \alpha_2 y_2 = 0$ gives $\alpha_1 = \alpha_2 = \alpha$:

$$\sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^\top \mathbf{x}_j = 2\alpha - \frac{1}{2}\cdot 2\,(2\alpha)^2 = 2\alpha - 4\alpha^2 \;\Rightarrow\; \alpha = \frac{1}{4}$$

$$\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i = \tfrac{1}{4}(1,1) + \tfrac{1}{4}(1,1) = (\tfrac{1}{2}, \tfrac{1}{2}) \quad \checkmark$$

Dual optimum $2 \cdot \frac{1}{4} - 4 \cdot \frac{1}{16} = \frac{1}{4}$ = primal optimum (strong duality); both points are support vectors.



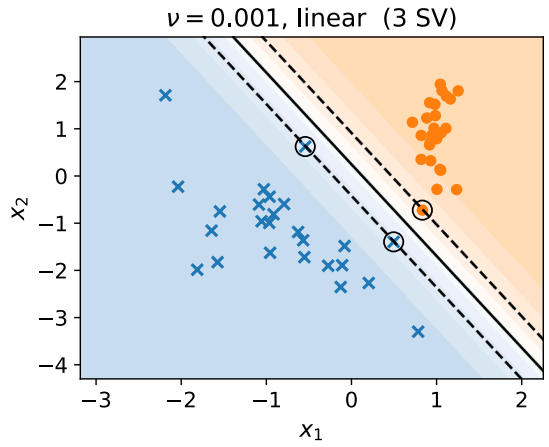
ν-SVM: why ν bounds the two fractions

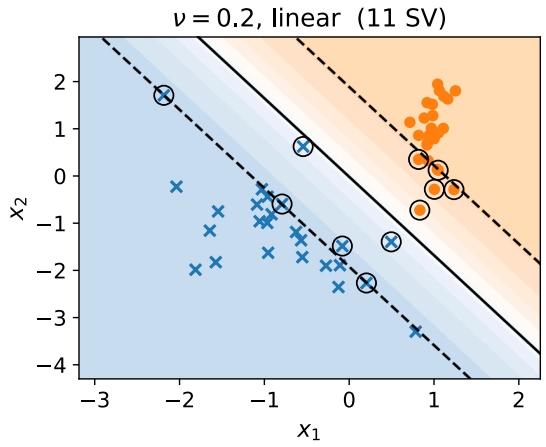
The primal of the main slide ($C > 0$ for the C-SVM; $\nu \in (0, 1], \rho \geq 0$ here). Its **dual** (Schölkopf et al., 2000):

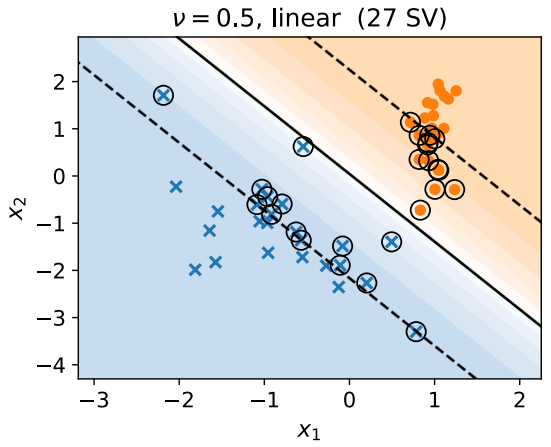
$$\max_{\boldsymbol{\alpha}} \; -\tfrac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \, \mathbf{x}_i^\top \mathbf{x}_j \quad \text{s.t.} \quad 0 \leq \alpha_i \leq \tfrac{1}{n}, \;\; \sum_i \alpha_i y_i = 0, \;\; \sum_i \alpha_i \geq \nu$$

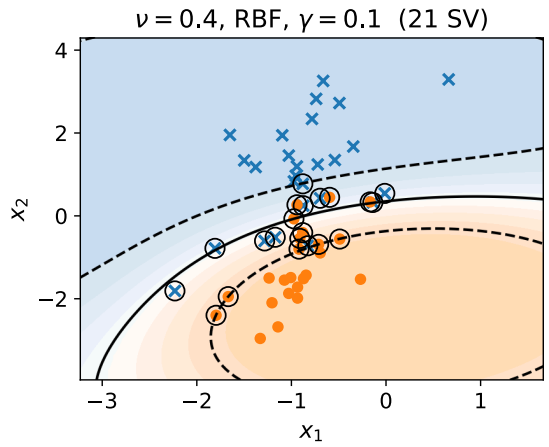
- When $\rho > 0$ at the optimum, the last constraint is active: $\sum_i \alpha_i = \nu$.
- **Margin error** ($\xi_i > 0$: inside the margin or misclassified) $\Rightarrow \alpha_i = \frac{1}{n}$ (KKT). So $\#\mathrm{ME} \cdot \frac{1}{n} \leq \sum_i \alpha_i = \nu$: at most a fraction ν of margin errors — and of training errors, which are margin errors too.
- **Support vector** $\Leftrightarrow \alpha_i > 0$, and every $\alpha_i \leq \frac{1}{n}$. So $\nu = \sum_i \alpha_i \leq \#\mathrm{SV} \cdot \frac{1}{n}$: at least a fraction ν of SVs.
- Dividing by ρ maps the solution to a C-SVM with $C = 1/(n\rho)$ — the same family of boundaries; with $\rho = 0$ (too small ν on overlapping data) the optimum is the trivial $\mathbf{w} = \mathbf{0}$, and the bounds say nothing.
- Kernels: replace $\mathbf{x}_i^\top \mathbf{x}_j$ by $k(\mathbf{x}_i, \mathbf{x}_j)$, as for the C-SVM.

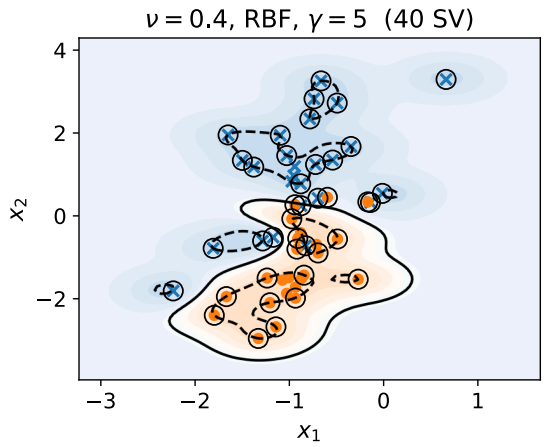
ν-SVM visualizations

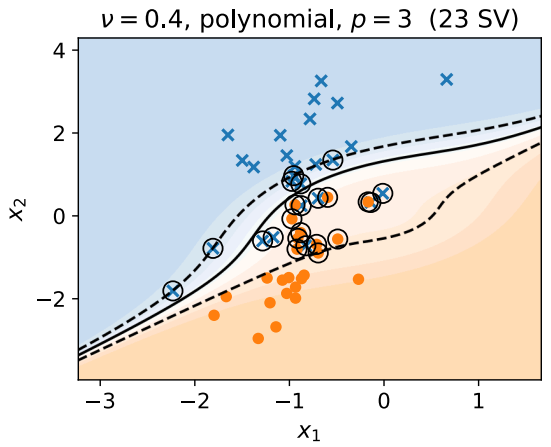












code example — visualization · code example — application